МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ПОВОЛЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра ИиСП

Отчет
по лабораторной работе № 5

по дисциплине «Машинно-зависимые языки программирования»
Вариант 4

Выполнил: ст. гр. ПС-13
Петялин А.Н.
Проверил: доцент кафедры ИиСП Баев А.А.

г. Йошкар-Ола
2025

Цель работы:

Научиться собирать схемы в симуляторе Proteus ISIS.

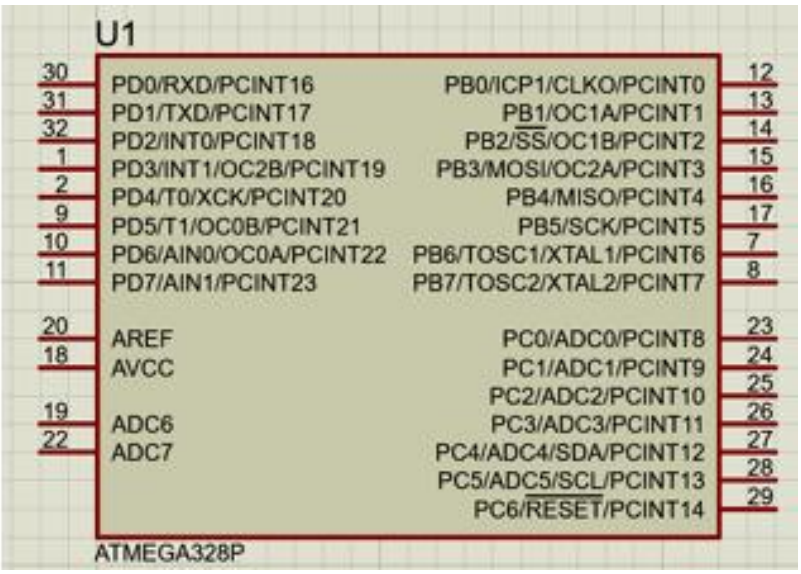
Задания на лабораторную работу:

1) Подключение светодиода
2) Подключение 7-сегментного индикатора
3) Подключение кнопки
4) Секундомер
5) Внешнее прерывание.

1. Теоретические сведения

Для отладки программ для контроллеров, не оснащенных JTAG интерфейсом, применяются методы компьютерного моделирования. Разработаны специализированные программные пакеты, которые позволяют смоделировать поведение электрических схем, в том числе и содержащих программируемые микросхемы. Одним из таких пакетов является Proteus. Данный пакет содержит в своем составе набор программ, позволяющих производить создание принципиальных схем, их моделирование, отладку и разработку печатных плат. В этой работе будет использован симулятор ISIS 7, входящий в данный пакет.

Для работы с симулятором необходимо собрать на рабочем поле схему устройства, работа которого будет моделироваться. Первоначально на рабочем поле размещается компонент контроллера. Это делается кликом правой клавиши мыши на свободном месте, в меню выбирается Place → Component → From libraries, где ищется ком понент под названием ATmega328, после чего он размещается на поле.



Для того чтобы Proteus мог моделировать поведение контроллера, необходимо передать в него hex-файл, содержащий программный код контроллера. В данном файле содержатся двоичные данные в таком виде, что они могут быть записаны и в реальный контроллер. Для получения данного файла необходимо написать и скомпилировать программу для контроллера.

Программа обычно пишется в сторонней среде программирования на языке С или Ассемблер, после чего с помощью компилятора получается необходимый hex файл.

Директива #include подключает к проекту файл io.h. Данный файл содержит в себе ссылки на другие файлы, подключаемые автоматически исходя из того, какой контроллер был выбран при создании проекта. В основном в этих файлах регистрам контроллера присваиваются определенные имена, что позволяет программисту с ними работать, иначе любая операция с внутренним содержимым контроллера потребовала бы от человека, занимающегося написанием ПО, прямого указания адресов в памяти контроллера, к которым он обращается. Эти имена являются общепринятыми для конкретного контроллера и приводятся в его техническом описании. Теперь можно заставить контроллер выполнить какое-либо действие. Пусть к определенному выводу контроллера через токоограничительный резистор подключен светодиод. Тогда, если подать напряжение на этот вывод, светодиод загорится. Для выполнения данного действия необходимо соответствующим образом настроить порты контроллера. Как уже отмечалось, выводы объединяются в порты, и доступ идет одновременно ко всему порту. С каждым портом связано не сколько управляющих регистров, задающих режим его работы. Эти регистры 8-битные, и каждый бит в них связан с соответствующим выводом порта.

DDRx (x – буква названия порта) – данный регистр задает режим работы порта: выводы используются либо как выходы, либо как логические входы. Если в определенном бите данного регистра стоит 1, то соответствующий вывод контроллера работает как логический выход, если 0 – как вход. Например, число 1810 = 1216 = 100102, записанное в регистр DDRB, настроит выводы PB1 и PB4 как выходы, а все остальные – как входы.

PORTx – функция данного регистра зависит от того, что записано в DDRx. Если соответствующий вывод настроен как выход, то запись 1 в соответствующий бит подаст на этот вывод напряжение. Если вывод настроен

на вход, то запись 1 включает подтягивающий к питанию резистор к этому выводу.

PINx – чтение данного регистра позволяет считать информацию с порта, но информативными будут лишь биты, соответствующие выводам, настроенным на вход.

Прерывание – сигнал, сообщающий процессору о наступлении какого-либо события. При этом выполнение текущей последовательности команд приостанавливается и управление передаётся обработчику прерывания, который реагирует на событие и обслуживает его, после чего возвращает управление в прерванный код. Прерывания могут быть вызваны различными событиями (как внешними для контроллера, так и внутренними). При поступлении сигнала прерывания контроллер приостанавливает работу с теку щей программой и сохраняет содержимое регистров ядра в специальной области памяти – в стеке, после чего переходит в область памяти, в которой расположена специальная подпрограмма – обработчик прерывания. По завершении работы обработчика контроллер восстанавливает содержимое регистров и продолжает работу с основной программой с того же места, на котором остановился. В данной реализации используется прерывание от внешнего источника – по изменению уровня напряжения на выводе с высокого на низкий. Для этого используется вывод PD2, так как альтернативной его функцией является работа в качестве источника внешнего прерывания INT0. Название обработчика данного прерывания – ISR(INT0_vect). Для использования данного прерывания необходимо особым образом настроить контроллер. Внешние прерывания настраиваются путем установки соответствующих бит в управляющие регистры контроллера внешних прерываний.

Регистр EICRA содержит контрольные биты для настройки условий генерации прерываний:

Bit	7	6	5	4	3	2	1	0	
0x1D (0x3D)	–	–	–	–	–	–	INT1	INT0	EIMSK
Read/write	R	R	R	R	R	R	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

При этом биты ISC11 и ISC10 отвечают за прерывание INT1, а ISC01 и ISC00 за INT0. Данные биты позволяют установить условия, при которых происходит генерация запроса на прерывание. Например, для прерывания INT0 возможны следующие комбинации:

ISC01	ISC00	Описание
0	0	Низкий логический уровень на INT0 вызывает прерывание
0	1	Любое изменение уровня на INT0 вызывает прерывание
1	0	Прерывание вызывается спадом сигнала на INT0
1	1	Прерывание вызывается фронтом сигнала на INT0

Для INT1 настройки аналогичны. Регистр EIMSK позволяет включить соответствующее прерывание:

Bit	7	6	5	4	3	2	1	0	
0x1D (0x3D)	–	–	–	–	–	–	INT1	INT0	EIMSK
Read/write	R	R	R	R	R	R	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

Для использования прерывания в программе необходимо вы полнить несколько условий:

1) подключить к проекту файл interrupt.h, в котором содержатся названия векторов обработчиков прерываний;

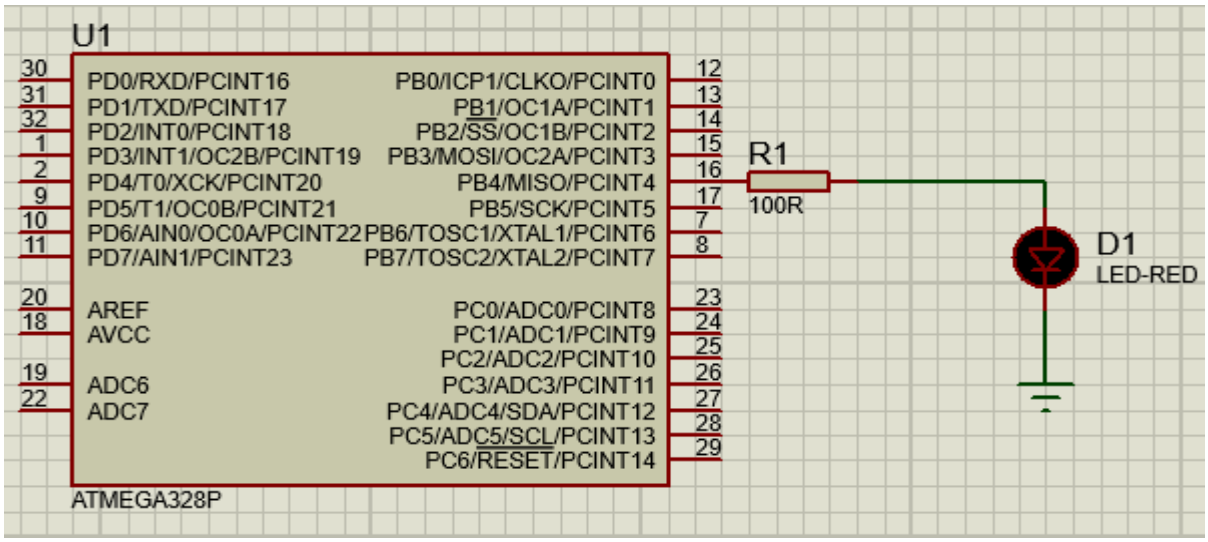
2) найти в описании периферийного устройства бит, установка которого разрешает генерацию запроса на прерывание, и включить его;

3) глобально разрешить использование прерываний в программе.

2. Практическая часть

1) Подключение светодиода

Для моделирования соберем в симуляторе Proteus ISIS следующую схему



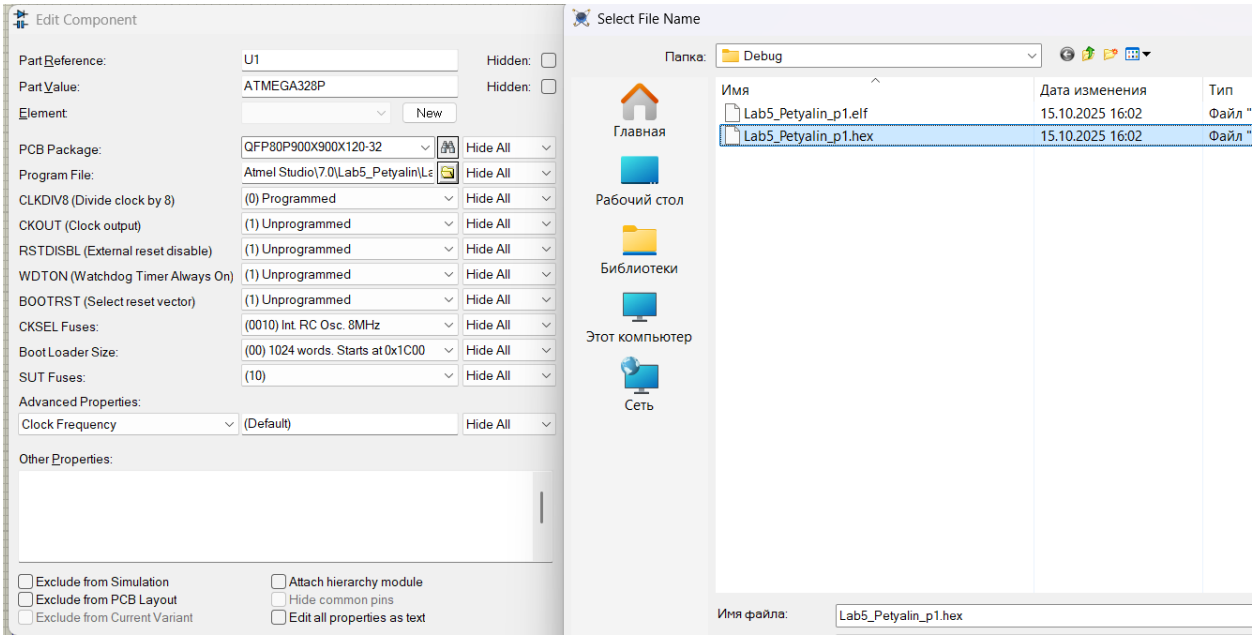
Светодиод подключен к выводу PB4. Исходя из этого, пишем следующий код:

```
#include <avr/io.h>


int main(void)
{
	DDRB = 0b00010000;
	PORTB = 0b00010000;
	while(1)
	{


	}
}
```

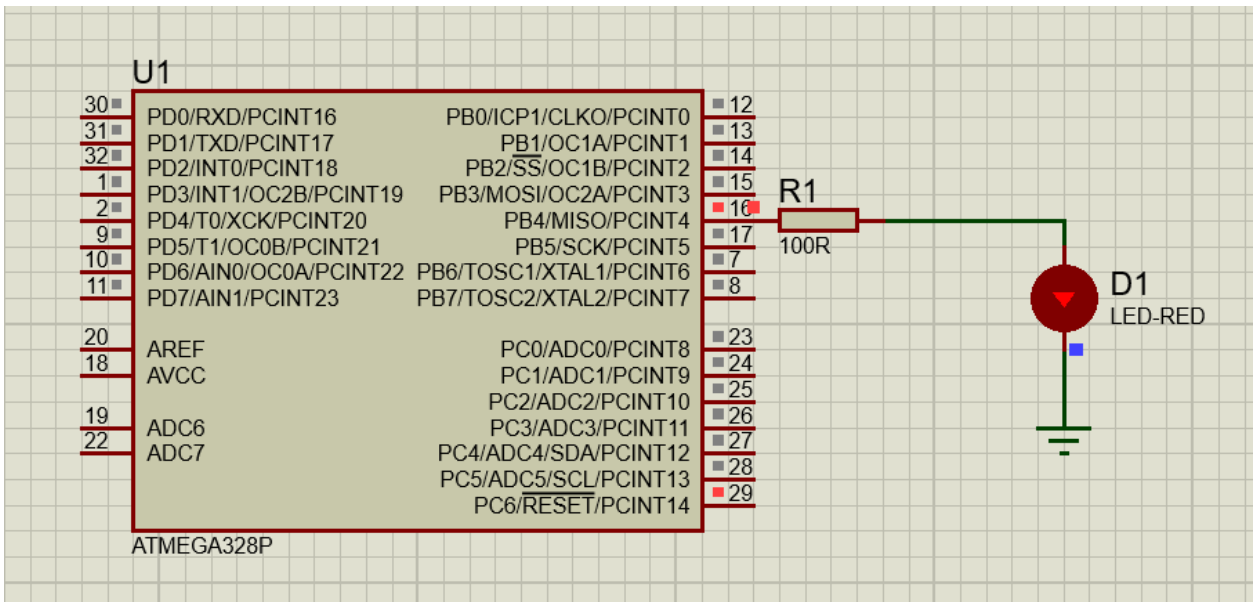
К шаблону добавилось всего 2 строки – в регистре DDRB в бит, соответствующий выводу PB4, записана 1, что настроило вывод как выход, а

в регистре PORTB единица в соответствующем бите подала на вывод напряжение. Можно отметить, что эти команды находятся вне цикла while, они будут выполнены только один раз – при старте контроллера. Обычно в этом месте располагают код, назначением которого является первоначальная настройка контроллера.

После того как код написан, необходимо сохранить и скомпилировать проект в Atmel Studio. Это делается из вкладки Build → Build solution. В результате в директории проекта появляется директория Debug, в которой, среди прочего, находится файл .hex. Этот файл и является конечной целью написания программы – в нем содержится написанная программа в виде машинных кодов, понятных контроллеру, и этот файл должен быть помещен в память контроллера. Поскольку для проверки работы программы используется симулятор Proteus ISIS, то в нем достаточно кликнуть правой кнопкой мыши на символе контроллера → Правка свойств и в графе Program file указать полученный hex-файл.



После этого можно запустить симуляцию, нажав на символ Play в левом нижнем углу экрана. Светодиод должен загореться.



Как видно из текста программы, запись числа в регистры сдела на в двоичном коде, однако это совершенно не обязательно. Форма представления данных не важна. Следующие команды идентичны между собой:

DDRB = 0b00010000; ; // двоичный код

DDRB = 0x10; // шестнадцатеричный код

DDRB = 16; // десятичное представление

DDRB = (1 << 4); // представление в виде сдвига

DDRB = (1 << PB4); // представление в виде сдвига

Отдельно следует отметить последние два представления. В языке С символами << и >> обозначаются операции битового сдвига влево и вправо соответственно. Так, запись 1<<4 означает сдвиг числа 1 на 4 позиции влево: то есть число 00000001_2 сдвигает ся 4 раза на 1 бит и получается число 00010000_2.

Во втором представлении используется название вывода PB4. Это наименование определено в файле iomx8.h, который подключен к проекту благодаря io.h. Он содержит строчку: #define PB4 4, которая означает, что при компиляции сочетание PB4 заменяется числом 4.

Подобная замена также может производиться и для других элементов, таких как адреса регистров, битовые маски и т.д. Такой под ход очень широко используется при программировании контроллеров, так как не только

избавляет программиста от необходимости держать в голове числовые значения, но и делает код более читабельным. Кроме того, отказ от использования конкретных чисел в коде имеет еще одно важное преимущество: он делает код независимым от контроллера, так как при смене контроллера становится до статочно сменить только заголовочные файлы, в которых содержатся конкретные определения используемых символов. Но чаще всего операции сдвига используются совместно с операторами &= и |=. Эти операции очень важны, так как позволяют изменять значения конкретных битов в числе. Это важно, когда необходимо изменить, например, значение одного вывода порта, не трогая остальные, при этом их значения могут быть и неизвестны. Например, запись вида

```
PORTB |= 1<<4;
```

может быть расписана как

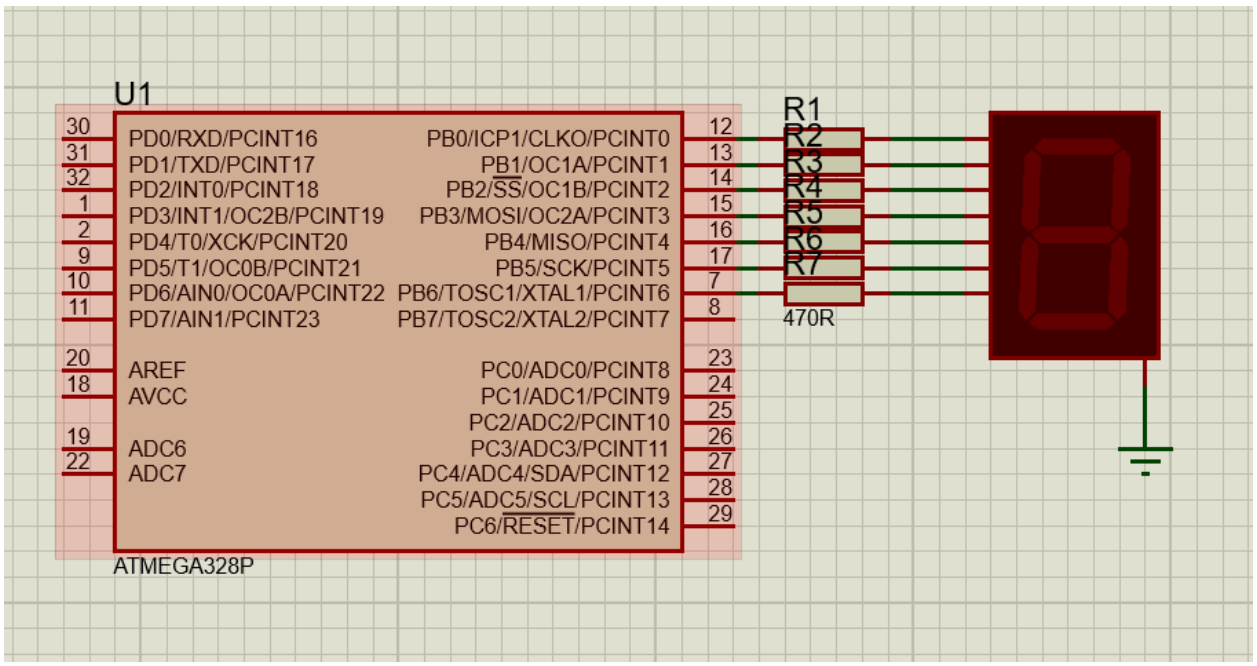
```
PORTB = PORTB | (1<<4);
```

что означает, что в PORTB будет записано значение PORTB, но по битово сложенное с числом 000100002, в результате бит 4 в PORTB станет равным 1, в то время как значение остальных не изменится. По аналогии, чтобы установить конкретный бит в 0 используется запись вида

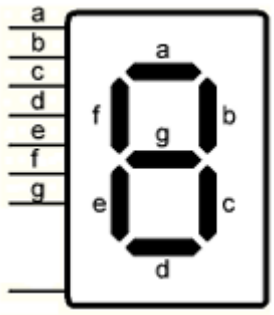
```
PORTB&= ~(1<<4);
```

2) Подключение 7-сегментного индикатора

Рассмотрим в Proteus ISIS следующую схему



Здесь к выводам порта B подключен 7-сегментный индикатор. В библиотеках Proteus ISIS этот индикатор располагается в категории Optoelectronics под названием 7SEG-COM-CATHODE. Данный индикатор представляет собой набор из 7 светодиодов, расположенных по известной схеме. Напряжение, поданное на один из выводов индикатора, зажигает соответствующий сегмент индикатора

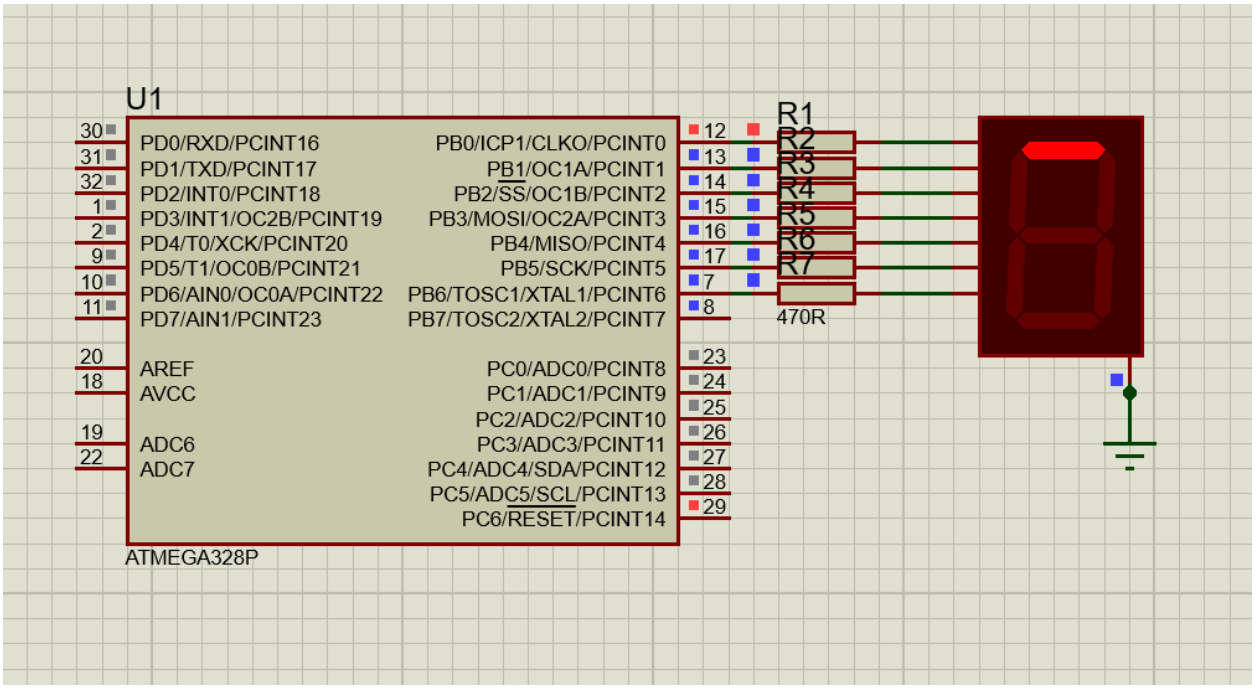


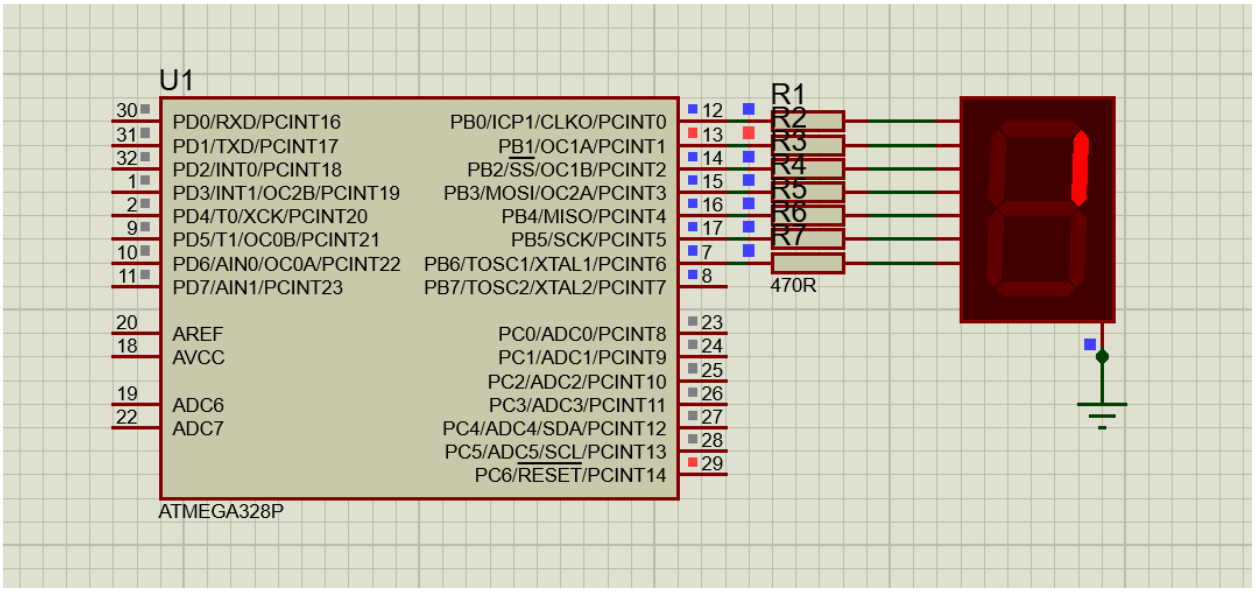
Приведенная ниже программа заставляет один светящийся сегмент индикатора перемещаться по кругу. Для этого необходимо настроить выводы порта B на выход и выводить на них соответствующий код. Однако так как контроллер при старте, в зависимости от заводских установок, работает на частоте от 1 до 8 МГц, то простое переключение выводов ничего не даст – частота изменения их состояния слишком велика, чтобы ее заметить. Необходимо использовать функцию задержки:

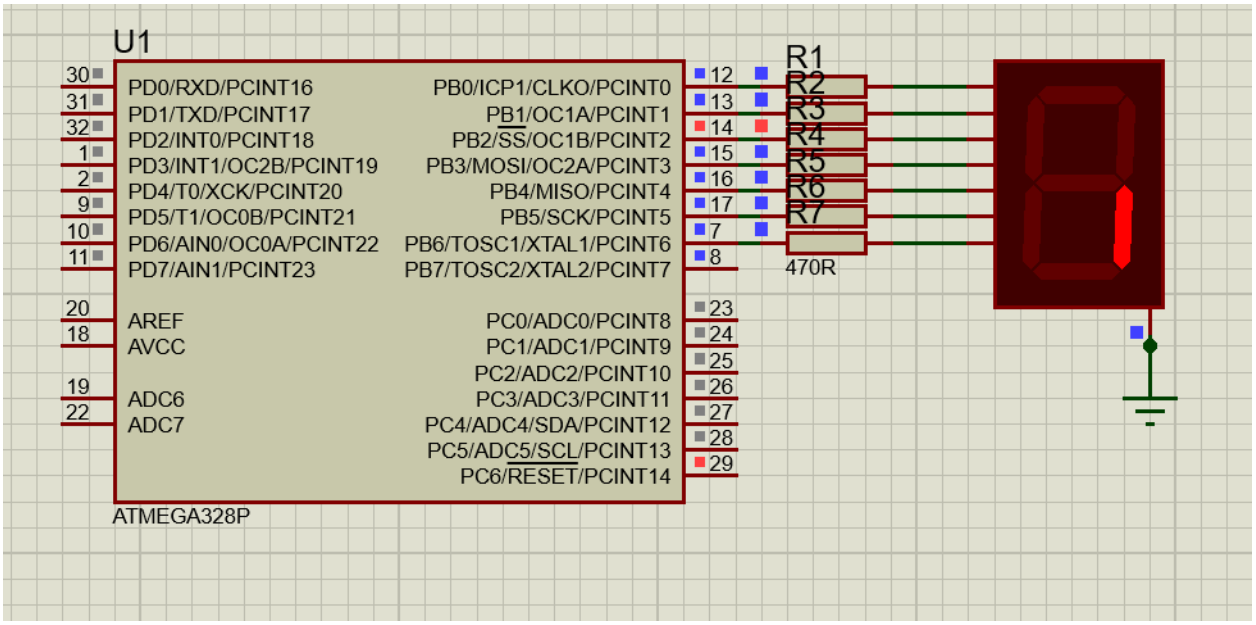
#define *F_CPU* 1000000UL

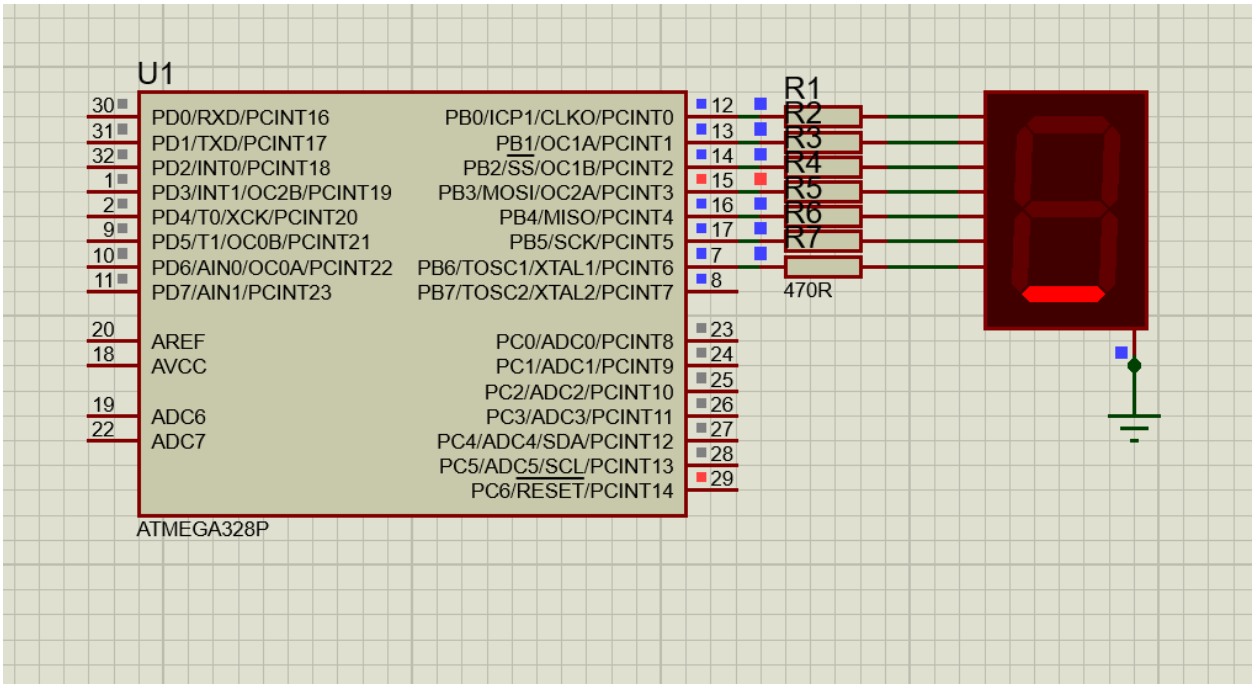
```
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
	DDRB = 0xFF;
	while(1)
	{
		for( int i = 0; i < 6; i++)
		{
			PORTB = (1 << i);
			_delay_ms(200);
		}
	}
}
```

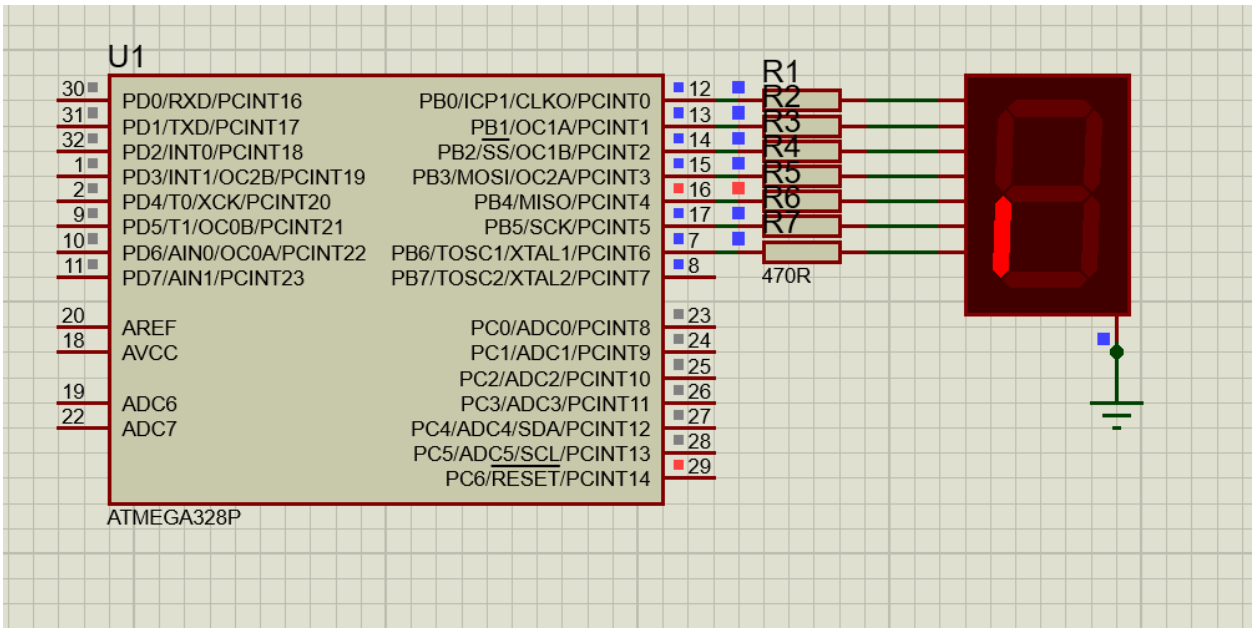
Здесь используется библиотечная функция _delay_ms(),определенная в файле delay.h. Для корректной 23 работы данной функции в проекте должна быть определена тактовая частота работы контроллера. Технически контроллер может работать на любой тактовой частоте, лишь бы она не была выше указанной в техническом описании, однако на практике есть ряд стандартных частот, которых стараются придерживаться. Директива #define F_CPU 1000000UL определяет тактовую частоту работы контроллера и необходима для работы функции задержки _delay_ms(), например: _delay_ms(200) - задержка на 200 мс. Необходимо отметить, что установка тактовой частоты таким образом в программе не отменяет необходимости в соответствующей установке фьюз-бит, а если установленные программно и аппаратно частоты будут отличаться, то все производимые в программе расчеты задержек будут выполнены неверно. В данной программе в порт B выводится 1, сдвинутая на число разрядов, заданное счетчиком цикла i – таким образом зажигаются сегменты от a до f. Когда цикл for кончается, наступает следующая итерация цикла while, и процесс повторяется.

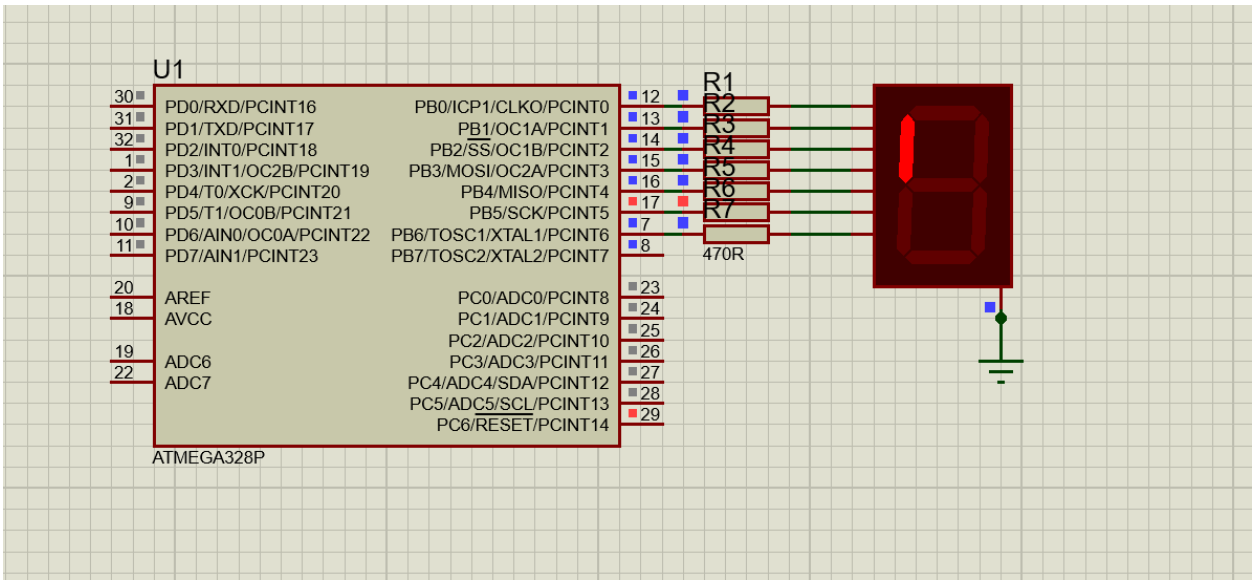

U1
30
31
32
1
2
9
10
11
20
18
19
22
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
12
13
14
15
16
17
7
8
23
24
25
26
27
28
29
R1
R2
R3
R4
R5
R6
R7
470R
ATMEGA328P


U1
30
31
32
1
2
9
10
11
20
18
19
22
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
12
13
14
15
16
17
7
8
23
24
25
26
27
28
29
R1
R2
R3
R4
R5
R6
R7
470R
ATMEGA328P


U1
30
31
32
1
2
9
10
11
20
18
19
22
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
12
13
14
15
16
17
7
8
23
24
25
26
27
28
29
R1
R2
R3
R4
R5
R6
R7
470R
ATMEGA328P

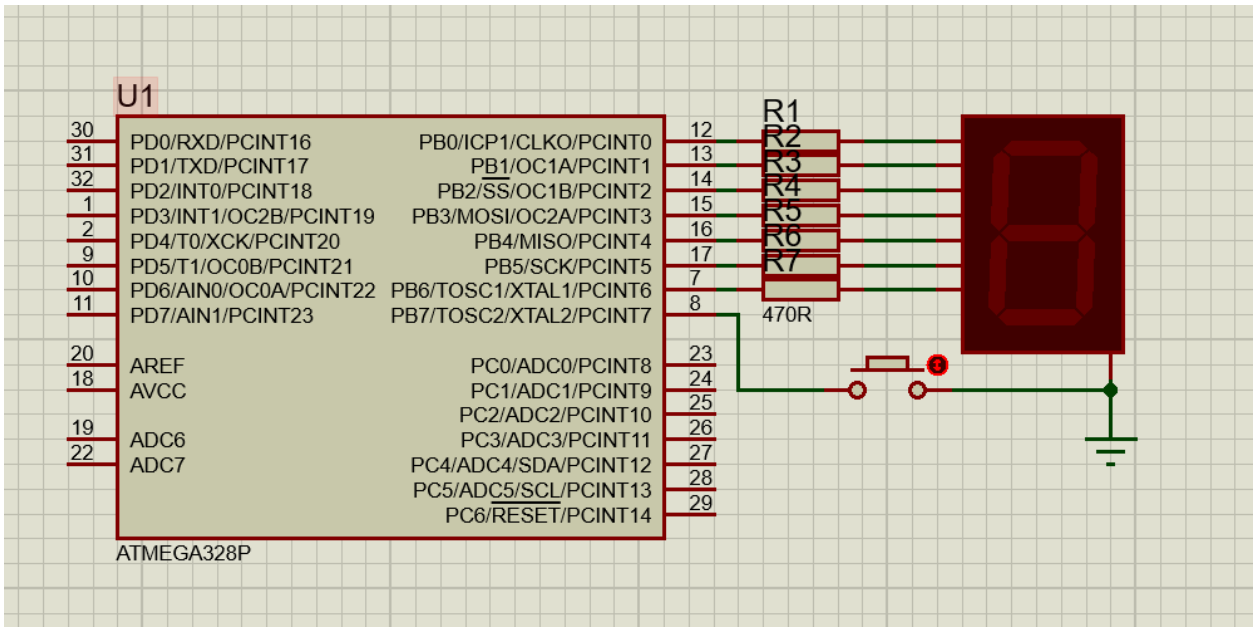

U1
30 PD0/RXD/PCINT16
31 PD1/TXD/PCINT17
32 PD2/INT0/PCINT18
1 PD3/INT1/OC2B/PCINT19
2 PD4/T0/XCK/PCINT20
9 PD5/T1/OC0B/PCINT21
10 PD6/AIN0/OC0A/PCINT22
11 PD7/AIN1/PCINT23
20 AREF
18 AVCC
19 ADC6
22 ADC7
PB0/ICP1/CLKO/PCINT0 12
PB1/OC1A/PCINT1 13
PB2/SS/OC1B/PCINT2 14
PB3/MOSI/OC2A/PCINT3 15
PB4/MISO/PCINT4 16
PB5/SCK/PCINT5 17
PB6/TOSC1/XTAL1/PCINT6 7
PB7/TOSC2/XTAL2/PCINT7 8
PC0/ADC0/PCINT8 23
PC1/ADC1/PCINT9 24
PC2/ADC2/PCINT10 25
PC3/ADC3/PCINT11 26
PC4/ADC4/SDA/PCINT12 27
PC5/ADC5/SCL/PCINT13 28
PC6/RESET/PCINT14 29
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R


U1
30 PD0/RXD/PCINT16
31 PD1/TXD/PCINT17
32 PD2/INT0/PCINT18
1 PD3/INT1/OC2B/PCINT19
2 PD4/T0/XCK/PCINT20
9 PD5/T1/OC0B/PCINT21
10 PD6/AIN0/OC0A/PCINT22
11 PD7/AIN1/PCINT23
20 AREF
18 AVCC
19 ADC6
22 ADC7
PB0/ICP1/CLKO/PCINT0 12
PB1/OC1A/PCINT1 13
PB2/SS/OC1B/PCINT2 14
PB3/MOSI/OC2A/PCINT3 15
PB4/MISO/PCINT4 16
PB5/SCK/PCINT5 17
PB6/TOSC1/XTAL1/PCINT6 7
PB7/TOSC2/XTAL2/PCINT7 8
PC0/ADC0/PCINT8 23
PC1/ADC1/PCINT9 24
PC2/ADC2/PCINT10 25
PC3/ADC3/PCINT11 26
PC4/ADC4/SDA/PCINT12 27
PC5/ADC5/SCL/PCINT13 28
PC6/RESET/PCINT14 29
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R


U1
30 PD0/RXD/PCINT16
31 PD1/TXD/PCINT17
32 PD2/INT0/PCINT18
1 PD3/INT1/OC2B/PCINT19
2 PD4/T0/XCK/PCINT20
9 PD5/T1/OC0B/PCINT21
10 PD6/AIN0/OC0A/PCINT22
11 PD7/AIN1/PCINT23
20 AREF
18 AVCC
19 ADC6
22 ADC7
PB0/ICP1/CLKO/PCINT0 12
PB1/OC1A/PCINT1 13
PB2/SS/OC1B/PCINT2 14
PB3/MOSI/OC2A/PCINT3 15
PB4/MISO/PCINT4 16
PB5/SCK/PCINT5 17
PB6/TOSC1/XTAL1/PCINT6 7
PB7/TOSC2/XTAL2/PCINT7 8
PC0/ADC0/PCINT8 23
PC1/ADC1/PCINT9 24
PC2/ADC2/PCINT10 25
PC3/ADC3/PCINT11 26
PC4/ADC4/SDA/PCINT12 27
PC5/ADC5/SCL/PCINT13 28
PC6/RESET/PCINT14 29
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R

3) Подключение кнопки

Тот же порт, к которому подключен индикатор, можно использовать одновременно и для подключения кнопки. Для этого схема изменяется следующим образом:

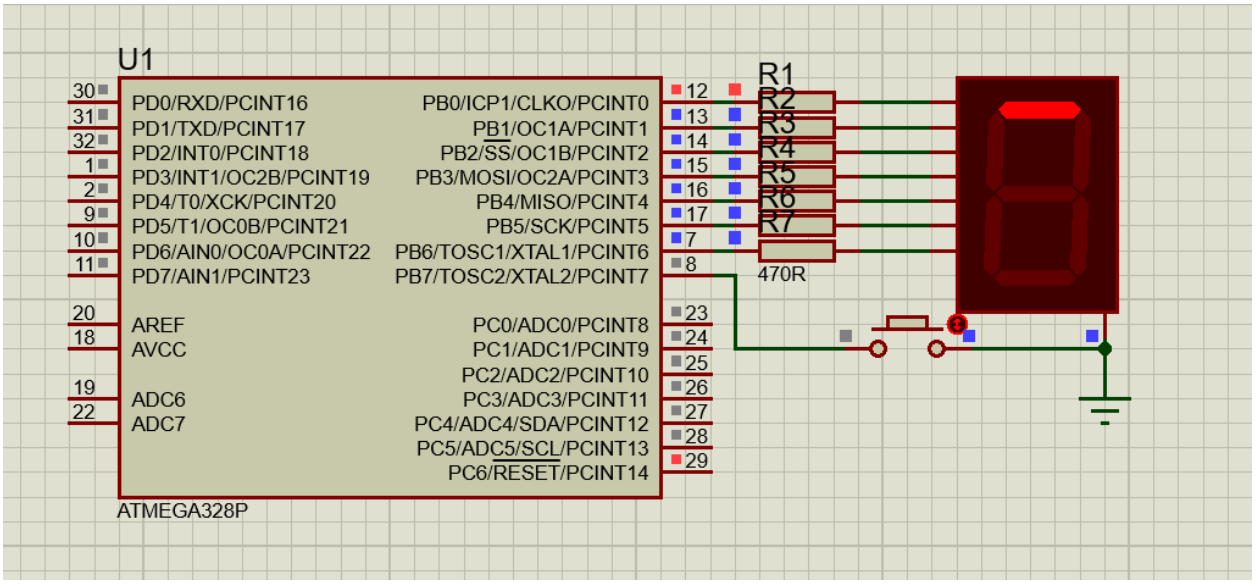


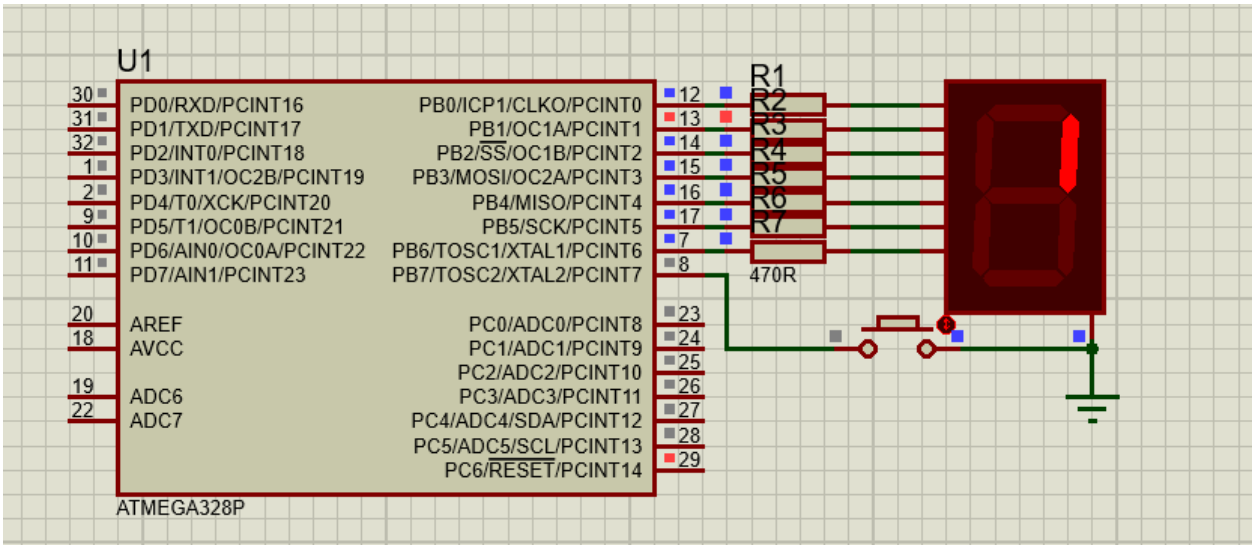
Кнопка подключена к выводу PB7. Для того чтобы использовать ее, необходимо сконфигурировать вывод PB7 как вход и включить внутренний

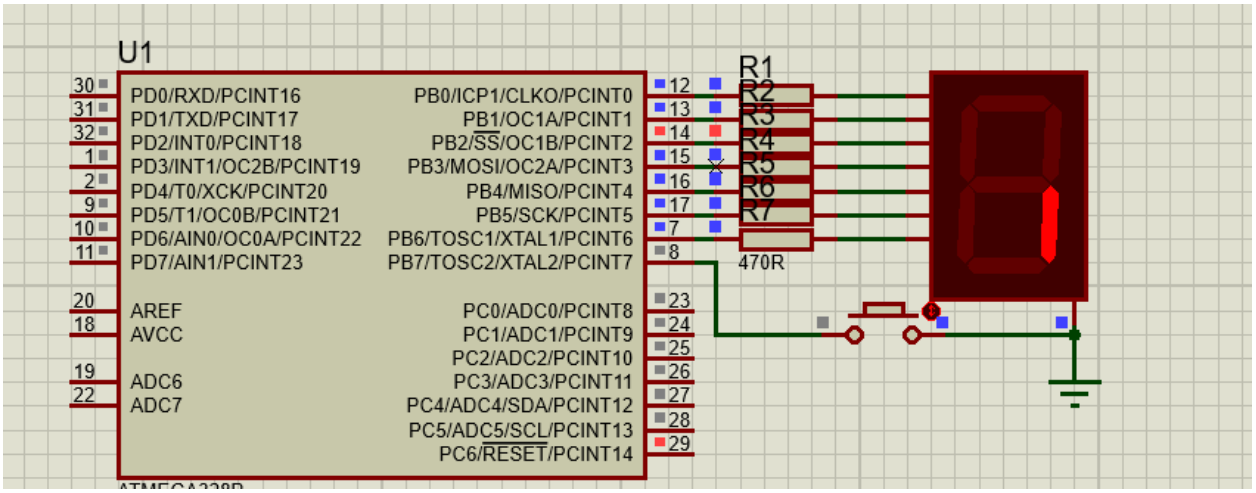
подтягивающий резистор – тогда в обычном состоянии с вывода будет считываться 1, а при нажатой кнопке – 0.

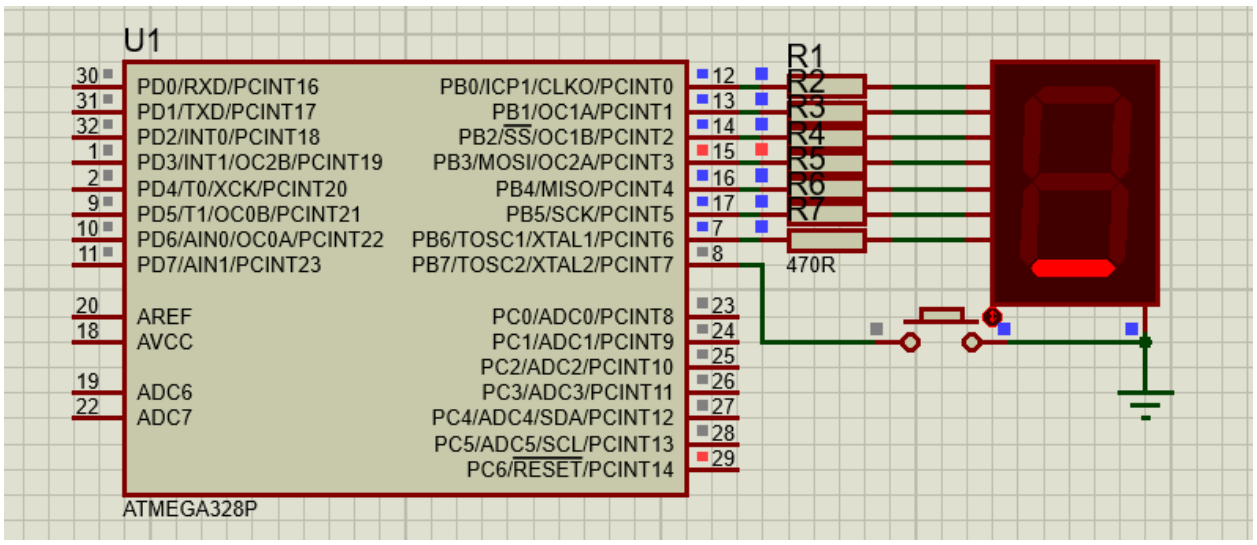
Программа модифицирована таким образом, что при удержании кнопки сегмент двигается в обратном направлении:

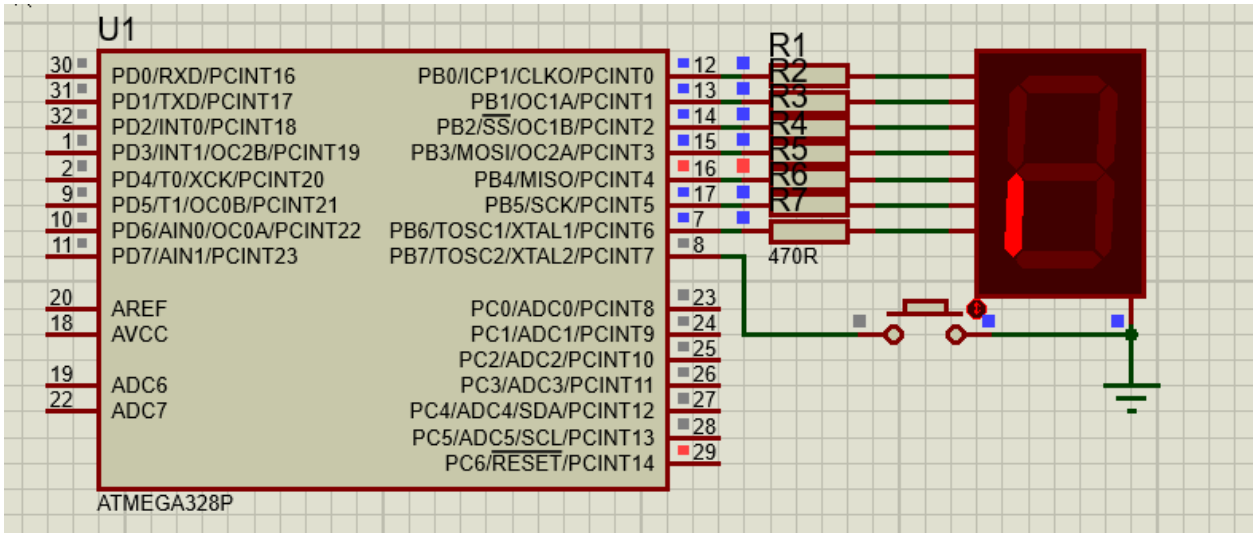
```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>
int main(void)
{
	DDRB = 0xFF & ~(1 << PINB7);// PB7 –  вход, остальные - выходы
	PORTB |= (1 << PINB7); // включение подтягивающего резистора
	int button = 0; //  вспомогательная переменная
	while(1)
	{
		for(int i = 0; i < 6; i++)
		{
			button = PINB & (1 << PINB7);
			// чтение состояния PB7
			if(button != 0)
			{
				PORTB = (1 << i);
				_delay_ms(200);
			}
			else
			{
				PORTB = (0x20 >> i);
				_delay_ms(200);
			}
		}
	}
}
```

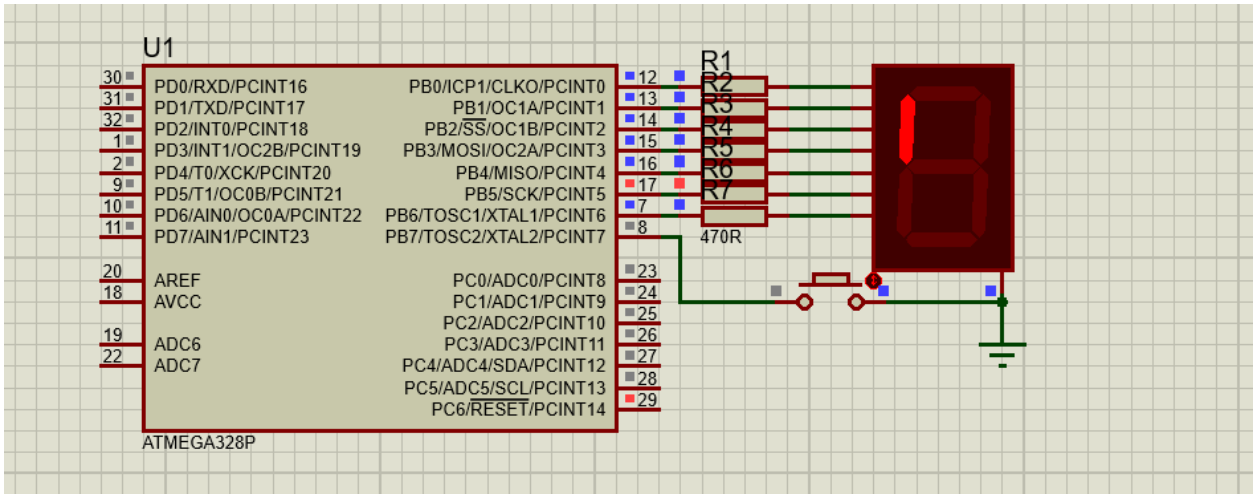

U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R


U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R

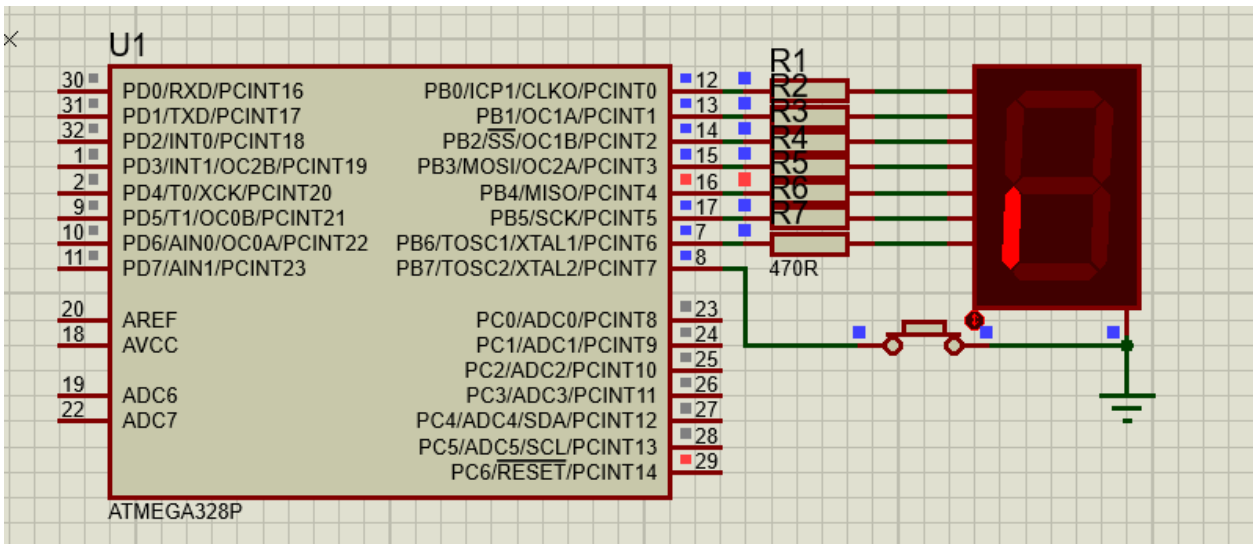

U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
R1
R2
R3
R4
R5
R6
R7
470R

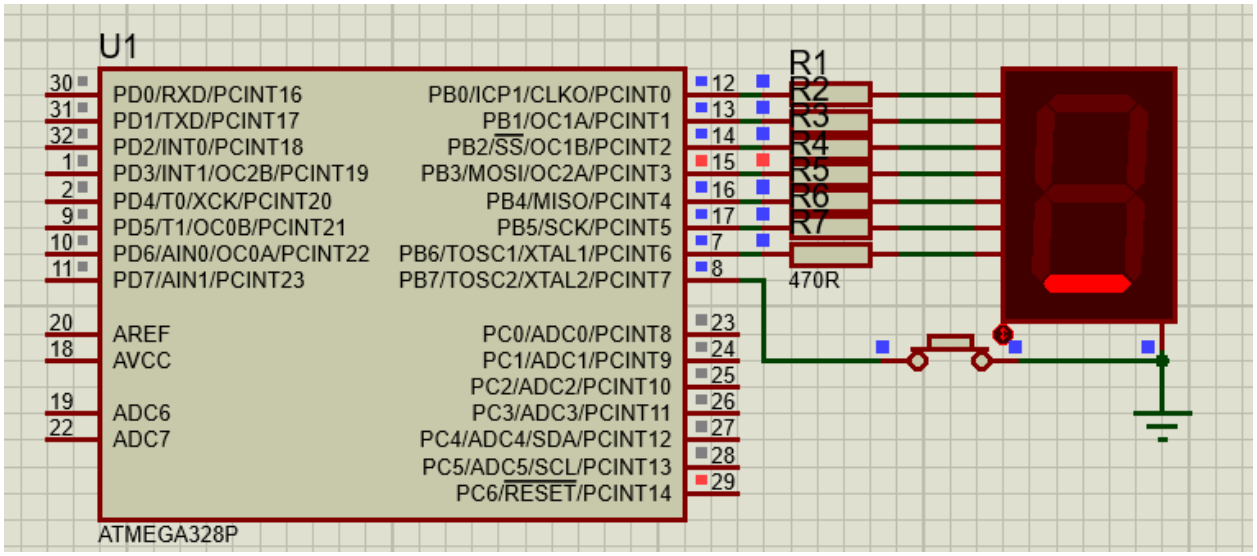


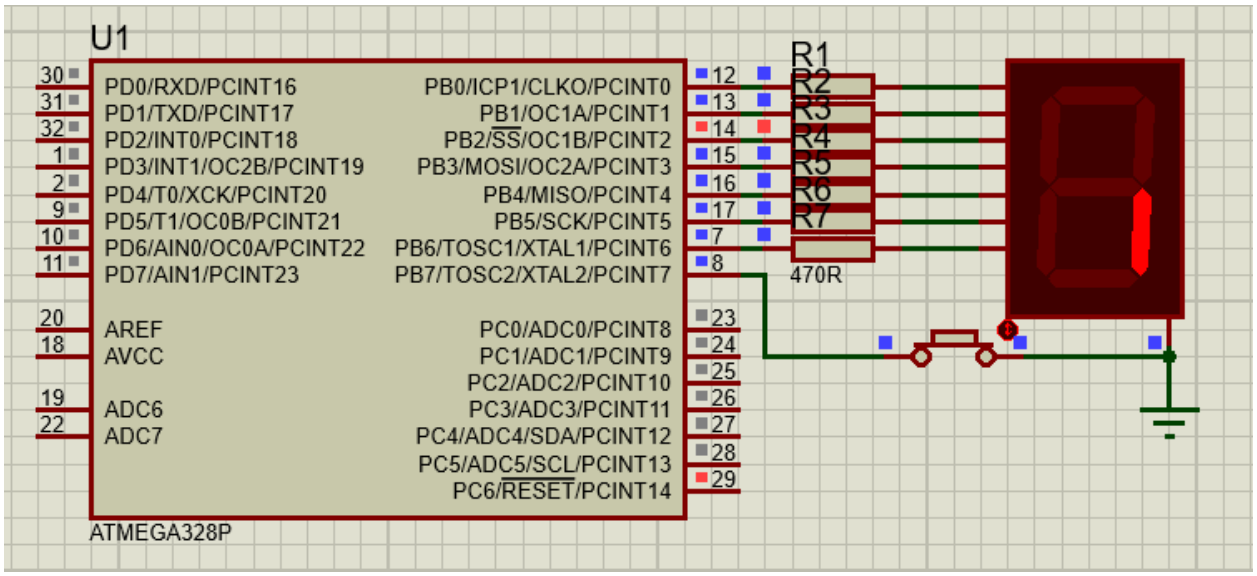


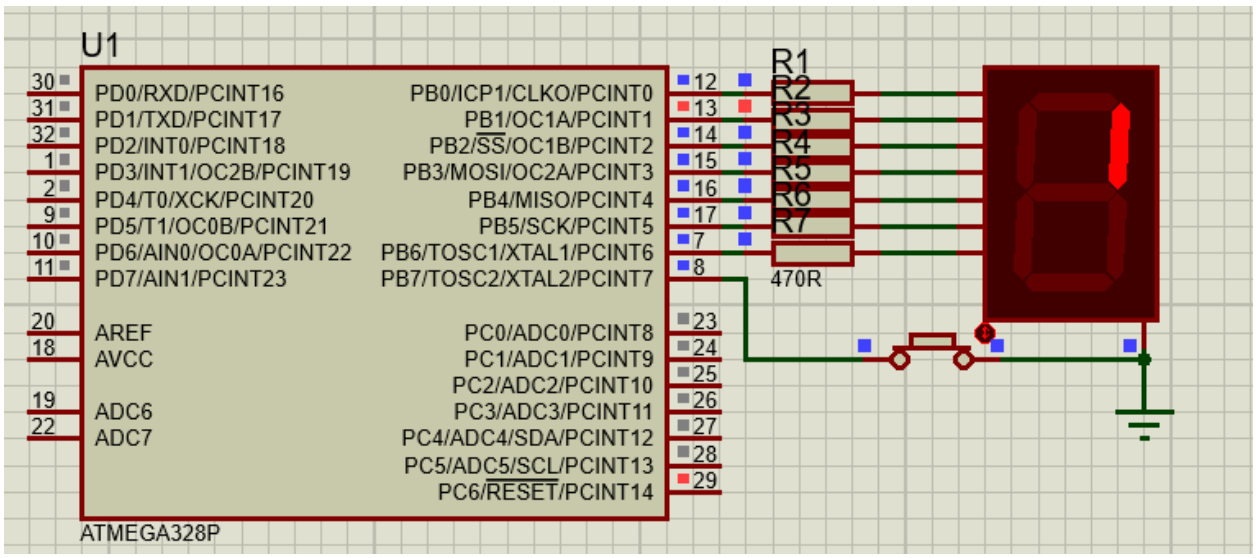


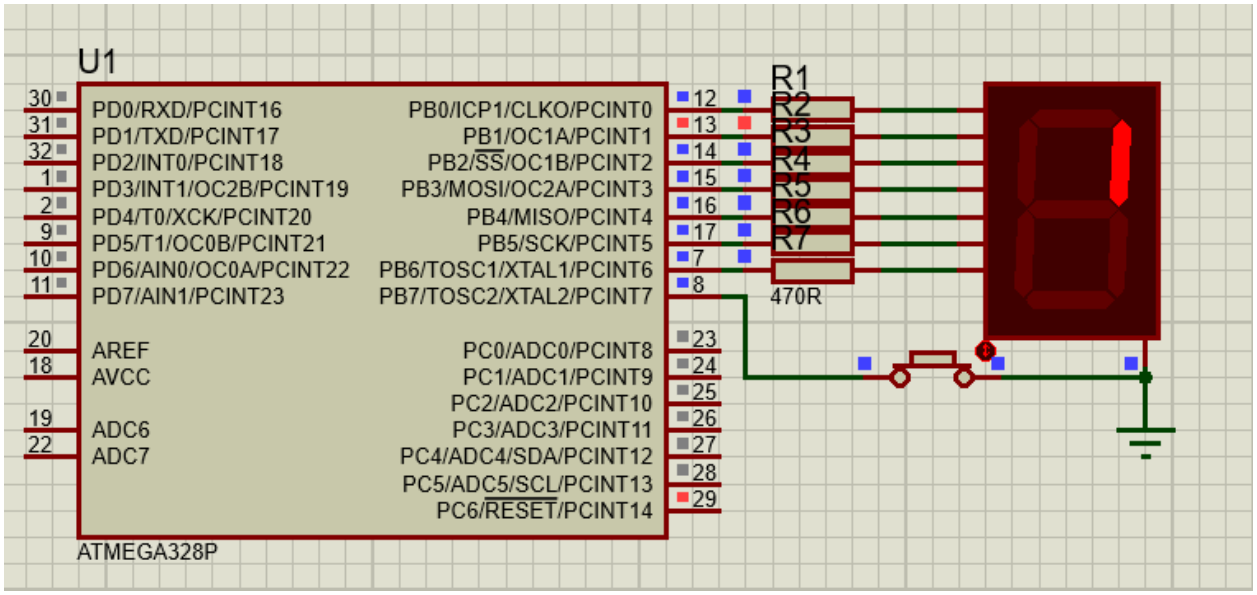
После зажатия кнопки:


U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R


U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R

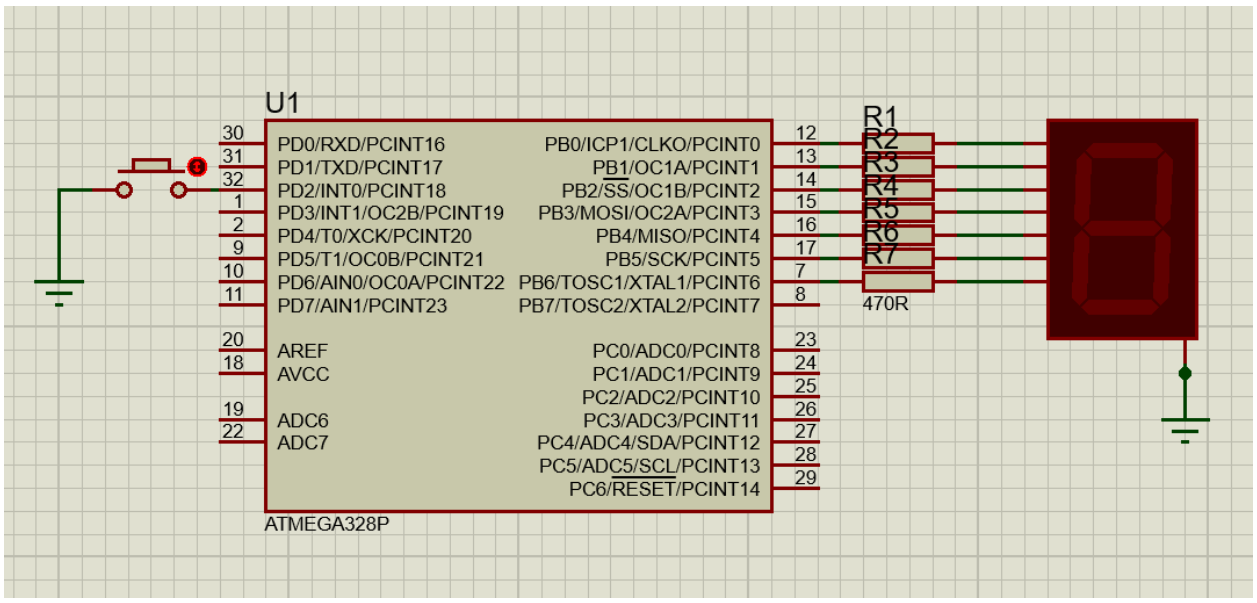

U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R





4) Секундомер

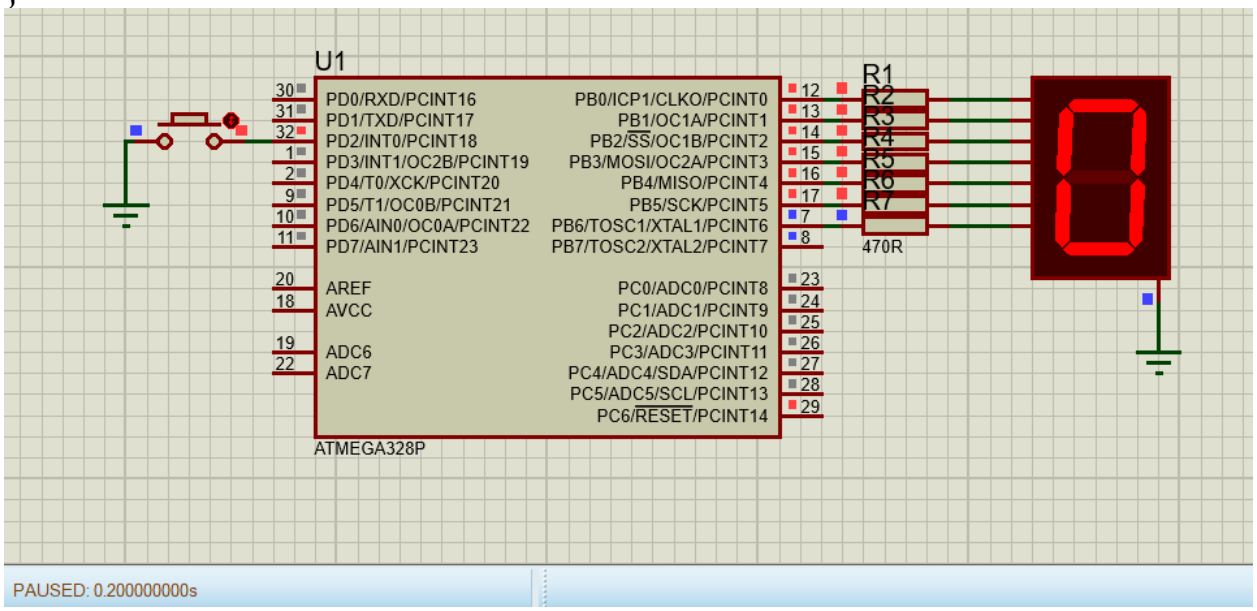
Семисегментный индикатор изначально предназначен для индикации цифр. Рассмотрим процесс создания секундомера на 10 секунд – по нажатию кнопки запускается и останавливается счет на индикаторе. Для индикации цифр создается массив чисел, каждое из которых соответствует набору горящих элементов индикатора, представляющих определенную цифру. Для упрощения процедуры записи в порт кнопка переносится на другой вывод, для того чтобы не было необходимости учитывать ее наличие. Полученная схема устройства представлена на рисунке.

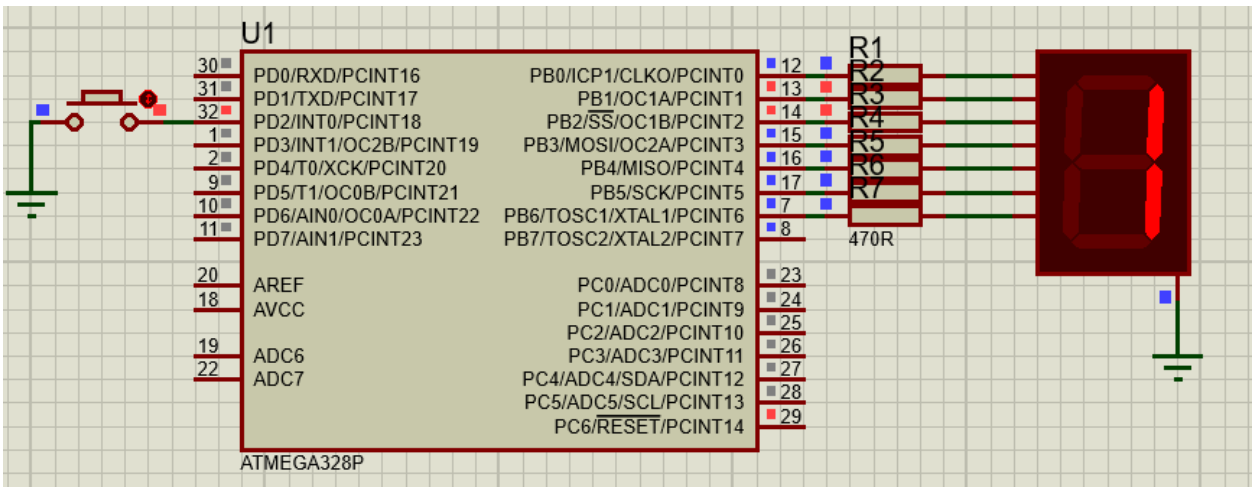


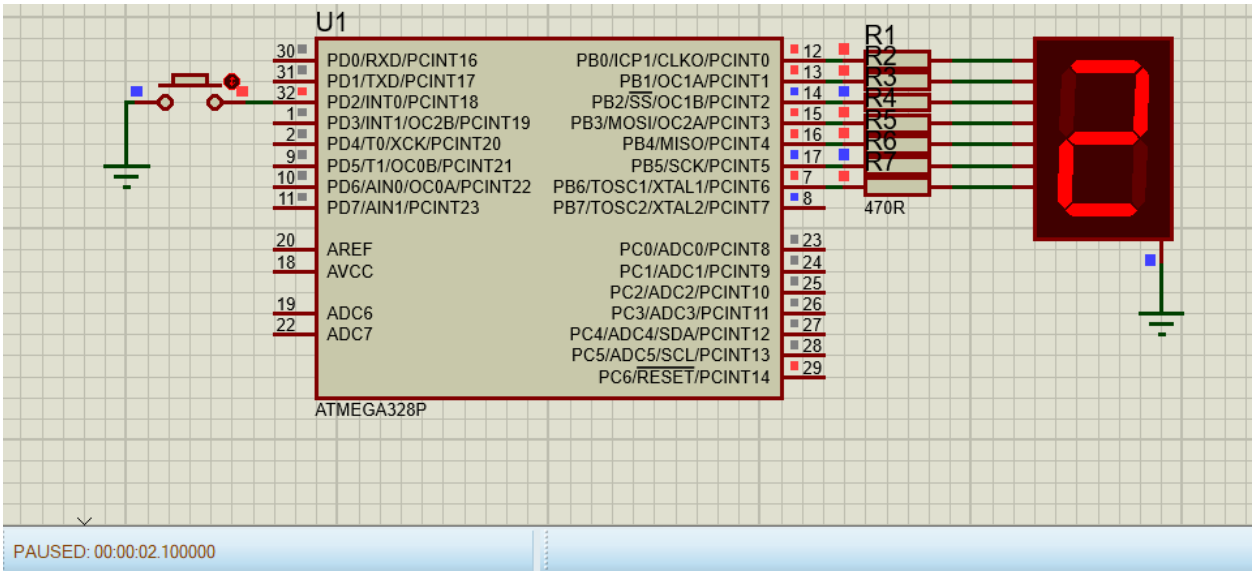
Программа:

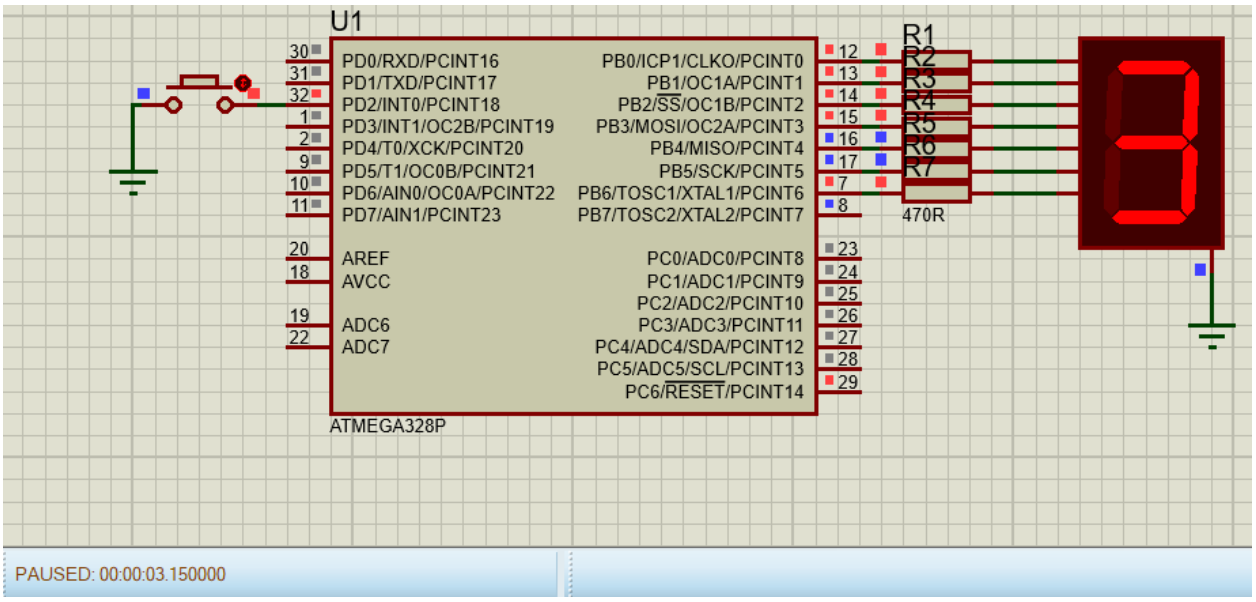
```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>

uint8_t segments[]=
{
	// ___GFEDCBA
	0b00111111, // 0 - A, B, C, D, E, F
	0b00000110, // 1 - B, C
	0b01011011, // 2 - A, B, D, E, G
	0b01001111, // 3 - A, B, C, D, G
	0b01100110, // 4 - B, C, F, G
	0b01101101, // 5 - A, C, D, F, G
	0b01111101, // 6 - A, C, D, E, F, G
	0b00000111, // 7 - A, B, C
	0b01111111, // 8 - A, B, C, D, E, F, G
	0b01101111, // 9 - A, B, C, D, F, G
};
int main(void)
{
	DDRB = 0xFF; // порт B на выход
	DDRD &= ~(1 << PIND2); // вывод PD2 на вход
	PORTD |= (1 << PIND2); // подтяжка PD2 включена
	int button = 0;
	int switch_state = 0;
	int counter = 0;
	while(1)
	{
		button = PIND & (1 << PIND2); //опрос кнопки
```

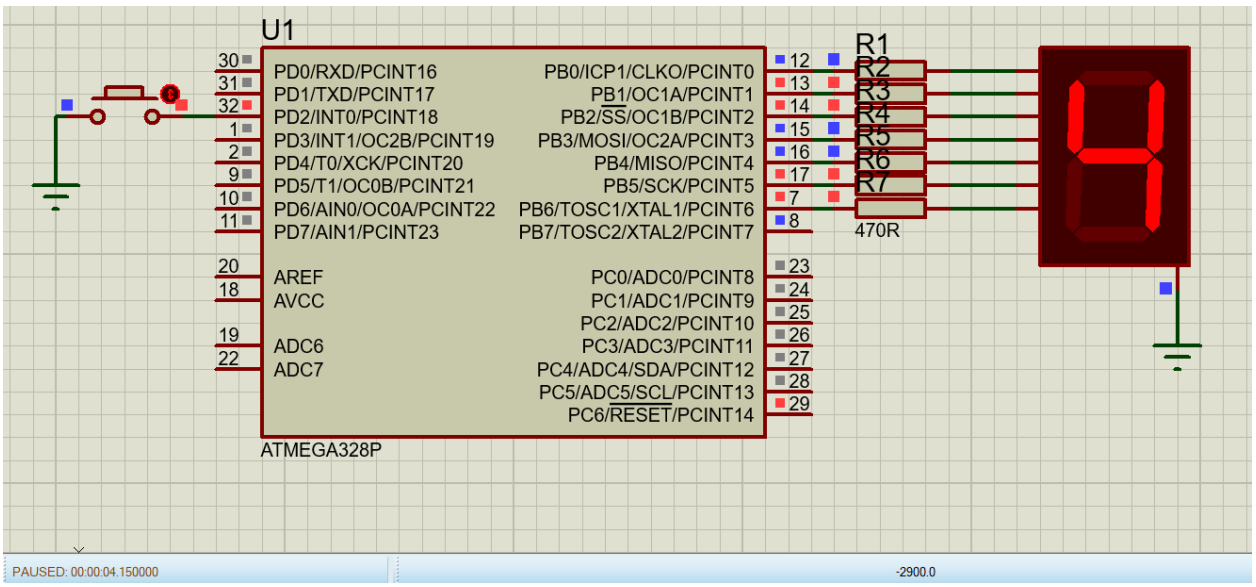
```
                if(button == 0)
                {
                        while((PIND & (1 << PIND2)) == 0);//ожидание
отпускания
                        if(switch_state == 0)
                        {
                                switch_state = 1;
                        }
                        else
                        {
                                switch_state = 0;
                                counter = 0;
                        }
                }
                if(switch_state == 0)
                {
                        if(counter < 10)
                        {
                                PORTB = segments[counter++];
                                _delay_ms(1000);
                        }
                        else
                        {
                                counter = 0;
                                PORTB = segments[counter++];
                                _delay_ms(1000);
                        }
                }
        }
}
```

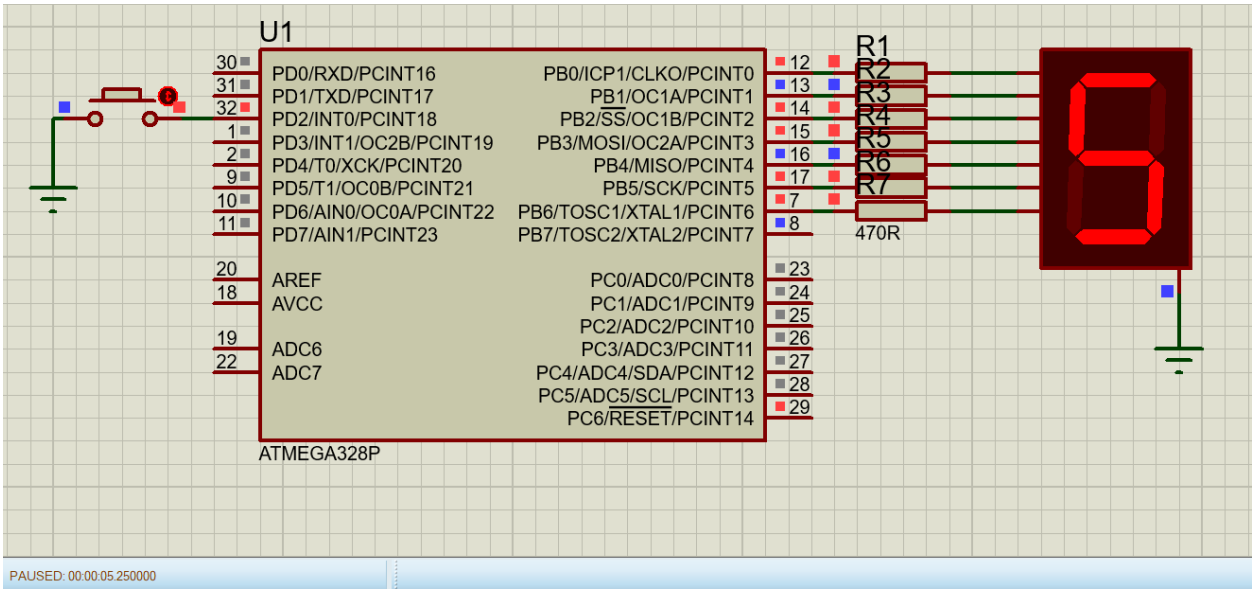


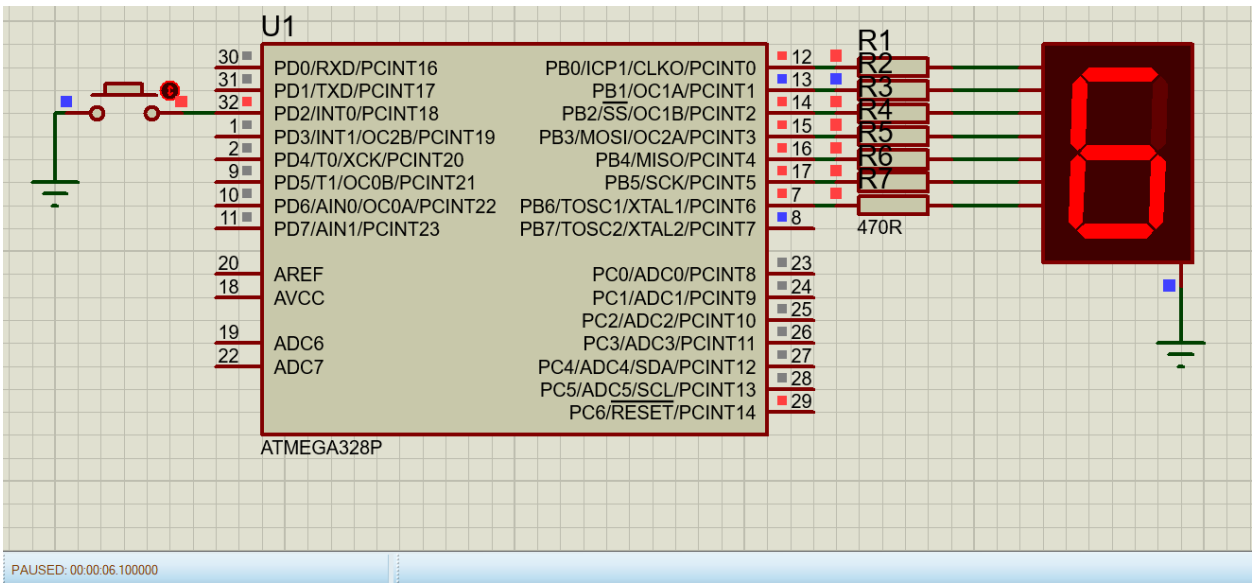

U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
470R

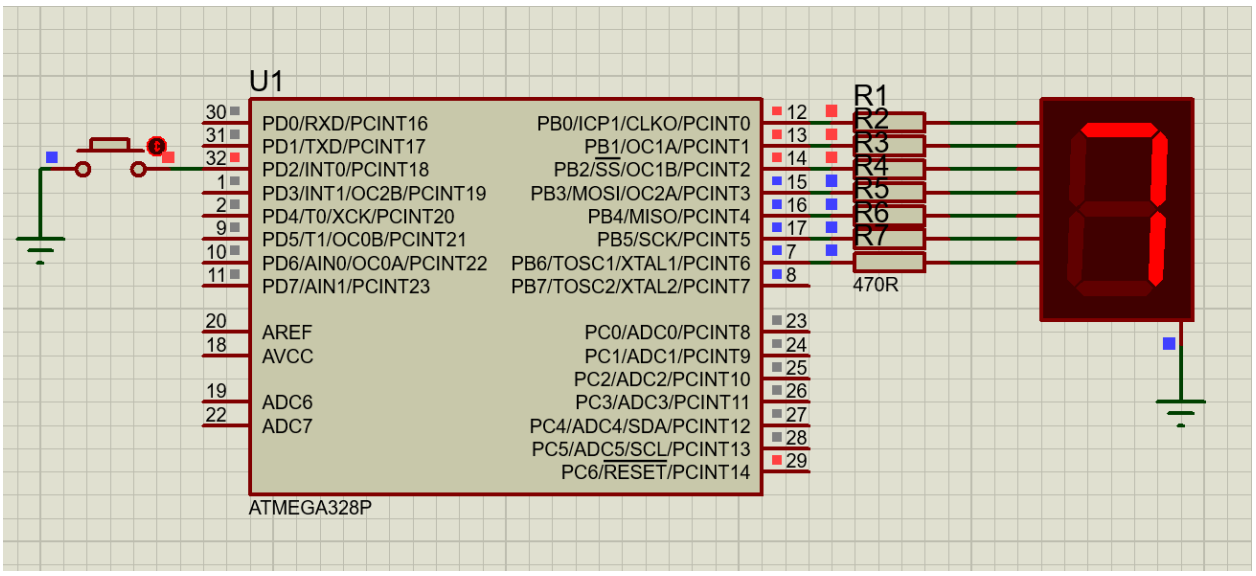

U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
470R
PAUSED: 00:00:02.100000

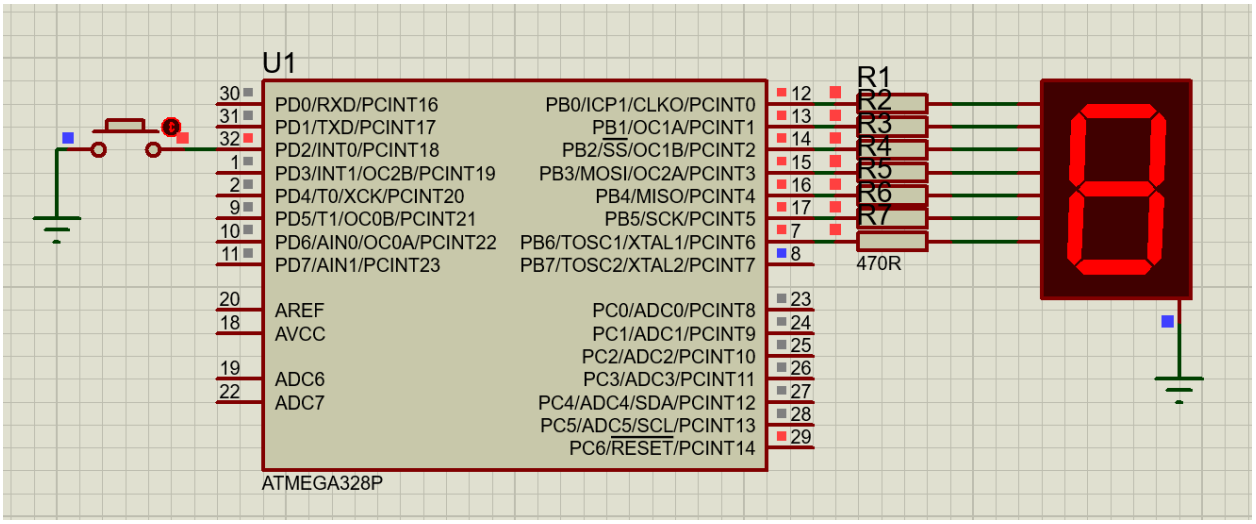

U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
470R
PAUSED: 00:00:03.150000

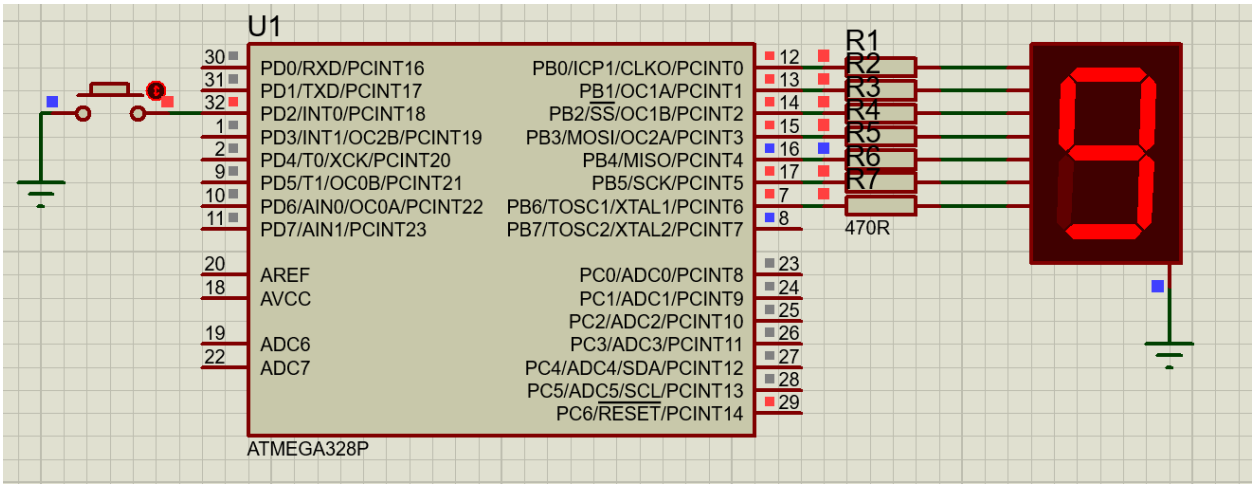

U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R
PAUSED: 00:00:04.150000
-2900.0

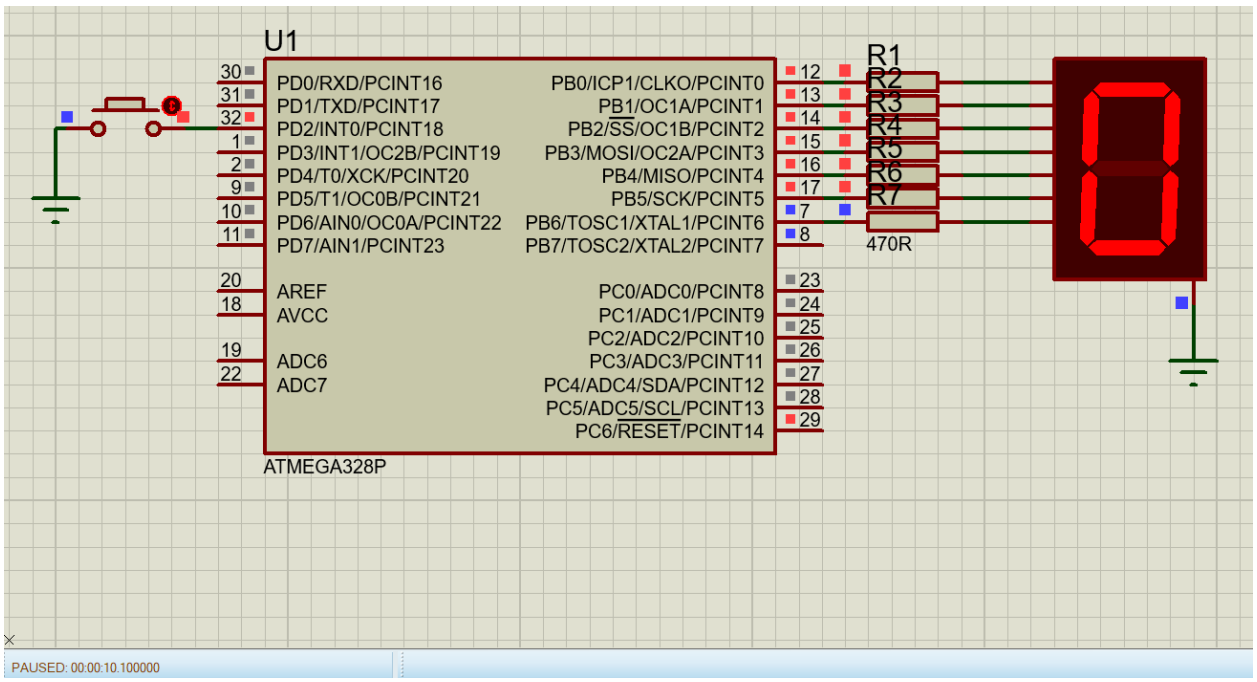

U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R
PAUSED: 00:00:05.250000


U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
R1
R2
R3
R4
R5
R6
R7
470R
PAUSED: 00:00:06.100000

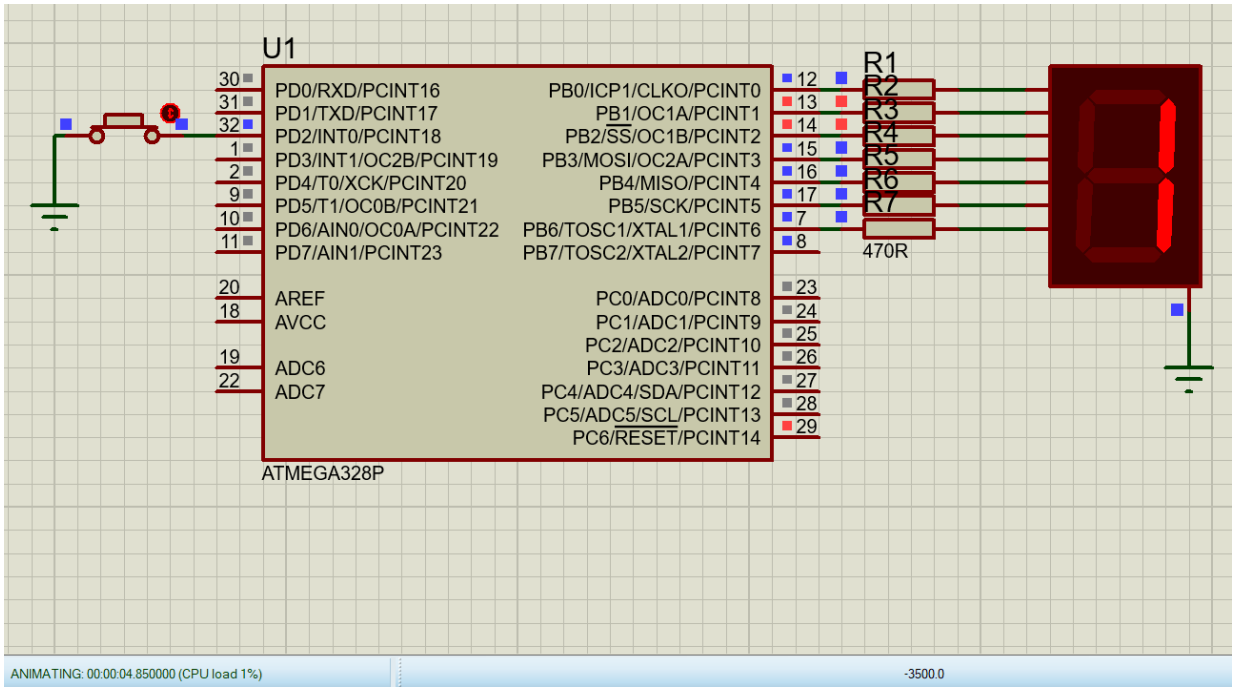

U1
30
31
32
1
2
9
10
11
20
18
19
22
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
12
13
14
15
16
17
7
8
23
24
25
26
27
28
29
R1
R2
R3
R4
R5
R6
R7
470R
ATMEGA328P


U1
30
31
32
1
2
9
10
11
20
18
19
22
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
12
13
14
15
16
17
7
8
23
24
25
26
27
28
29
R1
R2
R3
R4
R5
R6
R7
470R
ATMEGA328P

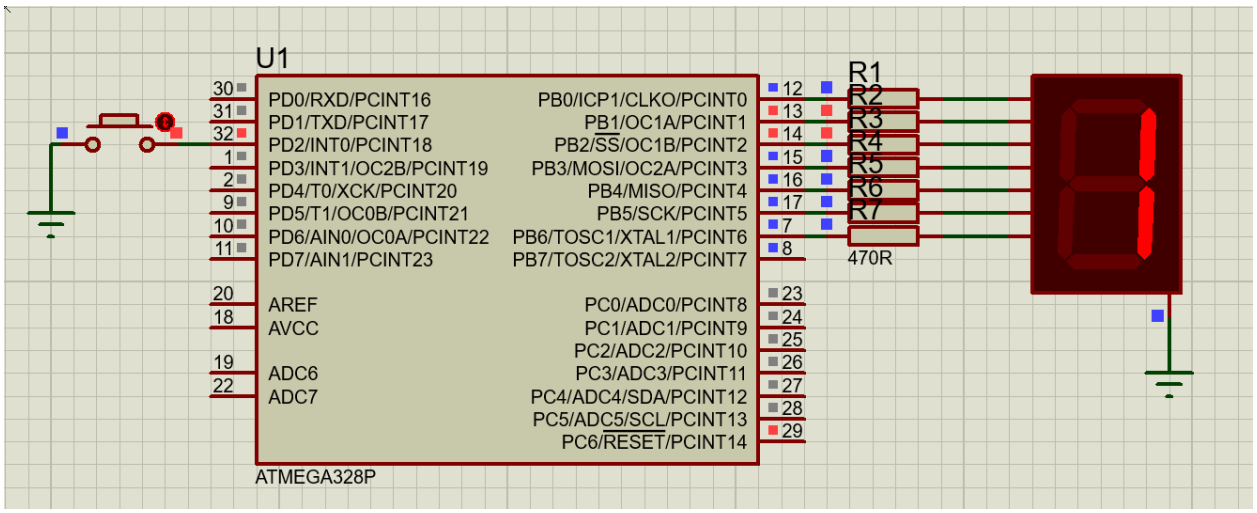

U1
30
31
32
1
2
9
10
11
20
18
19
22
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
12
13
14
15
16
17
7
8
23
24
25
26
27
28
29
R1
R2
R3
R4
R5
R6
R7
470R
ATMEGA328P



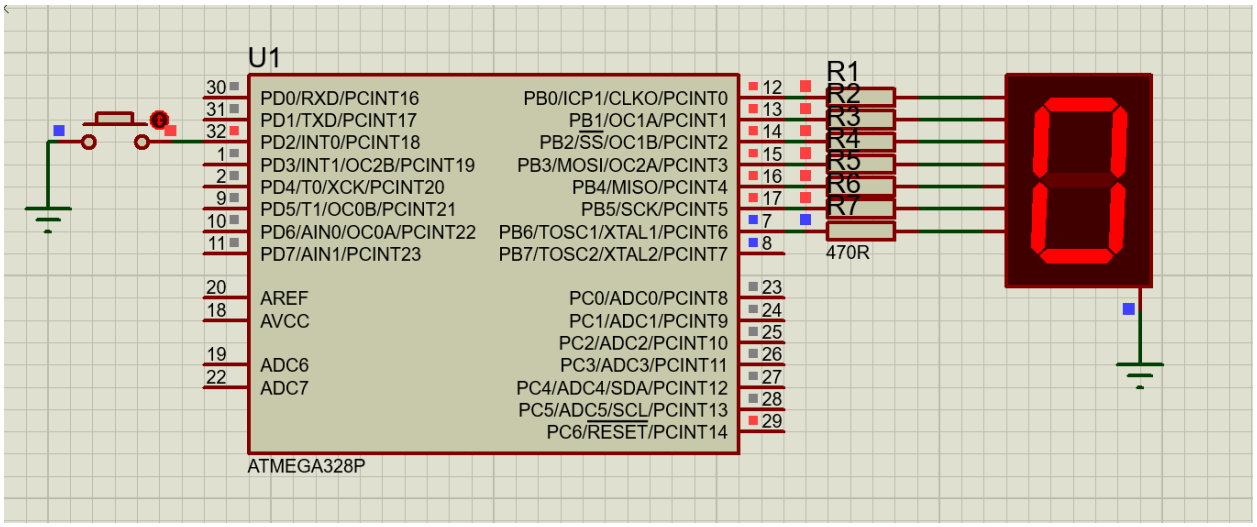
Когда кнопка зажата, отсчет останавливается:



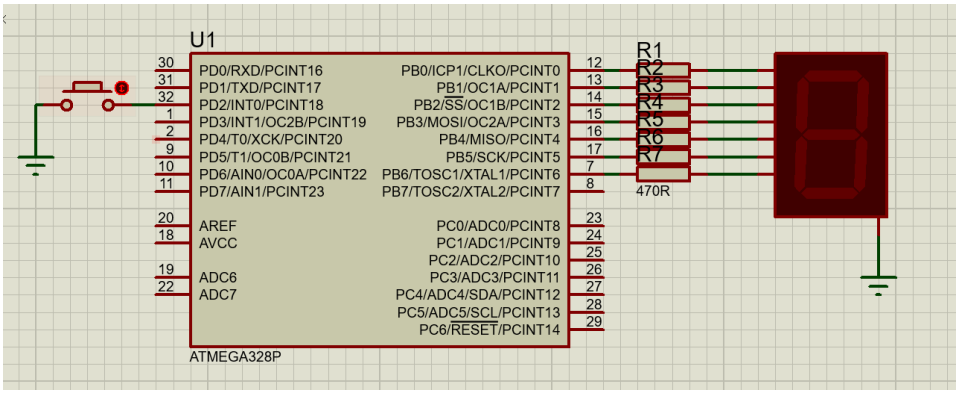
После отпускания кнопки ничего не происходит:



Для начала отсчета снова, нужно отпустить и нажать кнопку:



5) Внешнее прерывание.



При работе данной программы можно заметить, что кнопка плохо отзывается на нажатия. Дело в том, что опрос кнопки при работающем счете производится раз в секунду, так как ожидается, пока пройдет время задержки.

Во многих применениях подобная нечувствительность к срабатываниям неприемлема. В таком случае используется механизм прерываний.

Для использования вывода PD2 в качестве источника внешнего прерывания модуль INT0 настраивается на генерацию запроса на прерывание по спадающему фронту сигнала (переключение из 1 в 0). Это делается путем записи соответствующих значений в регистры EIMSK и EICRA. Конкретные значения приведены в техническом описании на контроллер.

Также необходимо глобально разрешить прерывания. Это делается короткой командой sei();

Также включается подтягивающий резистор на выводе PD2.

В обработчик прерывания выносится переключение состояния таймера и обнуление счетчика при повторном нажатии на кнопку.

Программа имеет следующий вид:

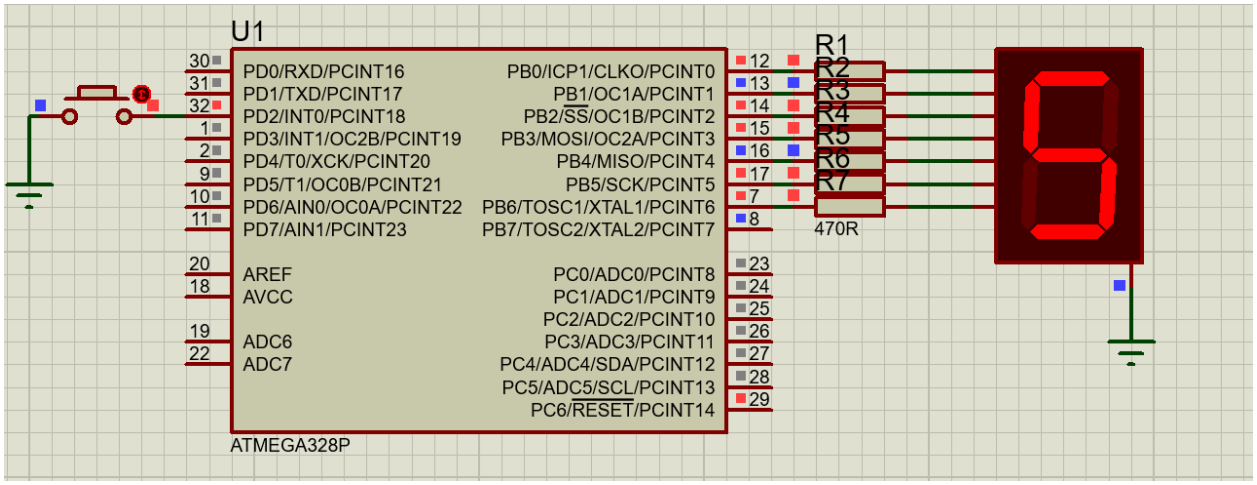
```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
uint8_t segments[]=
{ //0b01111111
	// ___GFEDCBA
	0b00111111, // 0 - A, B, C, D, E, F
	0b00000110, // 1 - B, C
	0b01011011, // 2 - A, B, D, E, G
	0b01001111, // 3 - A, B, C, D, G
	0b01100110, // 4 - B, C, F, G
	0b01101101, // 5 - A, C, D, F, G
	0b01111101, // 6 - A, C, D, E, F, G
	0b00000111, // 7 - A, B, C
	0b01111111, // 8 - A, B, C, D, E, F, G
	0b01101111, // 9 - A, B, C, D, F, G
};
volatile int button = 0;
volatile int switch_state = 0;
volatile int counter = 0;
ISR(INT0_vect)
{
	// Обработчик прерывания
	if(switch_state == 0)
```

```
		{
			switch_state = 1;
		}
		else
		{
			switch_state = 0;
			counter = 0;
		}
	}

	int main(void)
	{
		DDRB = 0xFF;
		PORTD |= (1 << PIND2);
		EIMSK |= (1 << INT0); //Включаем INT0
		EICRA |= (1 << ISC01); ///Прерывание по спадающему фронту
INT0
		sei();
		//Глобальное разрешение прерываний
		while(1)
		{
			if(switch_state == 0)
			{
				if(counter < 10)
				{
					PORTB = segments[counter];
					counter += 1;
					_delay_ms(500);
				}
				else
				{
					counter = 0;
					PORTB = segments[counter];
					counter += 1;
					_delay_ms(500);
				}
			}
		}
	}
```

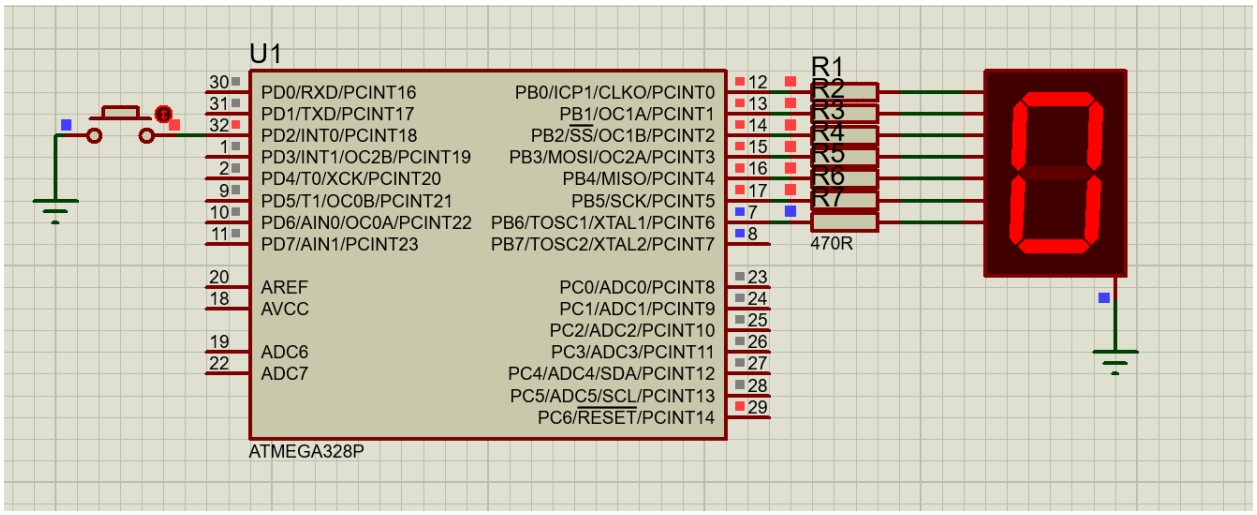
Теперь при включении секундомер начинает считать от нуля до девяти, а при нажатии на кнопку останавливается. При повторном нажатии переменная-счетчик обнуляется, и счет начинается с нуля.

Принцип работы примерно, как и в пункте 4, меняется только то, что опрос кнопки происходит мгновенно. Рассмотрим только разницу в отклике кнопки.

После нажатия кнопки отсчет мгновенно остановлен:



После повторного нажатия кнопки отсчет сразу же сбрасывается и начинается заново:



Оптимизация кодов:

Для первой и второй программ оптимизировать нечего.

Новый код для третьей программы:

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>
int main(void)
{
	DDRB = 0xFF & ~(1 << PINB7);
	PORTB |= (1 << PINB7);
	while(1)
	{
		for(uint8_t i = 0; i < 6; i++)
		{
			if(PINB & (1 << PINB7))
			{
				PORTB = (1 << i);
			}
			else
			{
				PORTB = (0x20 >> i);
			}
			_delay_ms(200);
		}
	}
}
```

Здесь я изменил тип у переменной i, чтобы она не занимала 2 регистра, убрал переменную button, чтобы не производить лишнего присваивания, вынес функцию задержки _delay_ms за скобки, т.к. она будет срабатывать вне зависимости от того, истинно условие или ложно.

Новый код для четвертой программы:

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>

uint8_t segments[]=
{
	// ___GFEDCBA
	0b00111111, // 0 - A, B, C, D, E, F
```

```
        0b00000110, // 1 - B, C
        0b01011011, // 2 - A, B, D, E, G
        0b01001111, // 3 - A, B, C, D, G
        0b01100110, // 4 - B, C, F, G
        0b01101101, // 5 - A, C, D, F, G
        0b01111101, // 6 - A, C, D, E, F, G
        0b00000111, // 7 - A, B, C
        0b01111111, // 8 - A, B, C, D, E, F, G
        0b01101111, // 9 - A, B, C, D, F, G
};
int main(void)
{
        DDRB = 0xFF;
        DDRD &= ~(1 << PIND2);
        PORTD |= (1 << PIND2);
        uint8_t switch_state = 0;
        uint8_t counter = 0;
        while(1)
        {
                if(!(PIND & (1 << PIND2)))
                {
                        while((PIND & (1 << PIND2)) == 0);
                        if(switch_state == 0)
                        {
                                switch_state = 1;
                        }
                        else
                        {
                                switch_state = 0;
                                counter = 0;
                        }
                }
                if(switch_state == 0)
                {
                        if(counter == 10)
                        {
                                counter = 0;
                        }
                        PORTB = segments[counter++];
                        _delay_ms(1000);
                }
        }
}
```

В этом коде я также изменил тип переменных, чтобы они занимали один регистр, а не два; также убрал переменную button, вынес функцию задержки за скобки и вынес вывод числа, т.к. в любом случае мы будем выводить что-то на индикатор.

Новый код для пятой программы:

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
uint8_t segments[]=
{ //0b01111111
	// ___GFEDCBA
	0b00111111, // 0 - A, B, C, D, E, F
	0b00000110, // 1 - B, C
	0b01011011, // 2 - A, B, D, E, G
	0b01001111, // 3 - A, B, C, D, G
	0b01100110, // 4 - B, C, F, G
	0b01101101, // 5 - A, C, D, F, G
	0b01111101, // 6 - A, C, D, E, F, G
	0b00000111, // 7 - A, B, C
	0b01111111, // 8 - A, B, C, D, E, F, G
	0b01101111, // 9 - A, B, C, D, F, G
};
volatile uint8_t switch_state = 0;
volatile uint8_t counter = 0;
ISR(INT0_vect)
{
	if(switch_state == 0)
	{
		switch_state = 1;
	}
	else
	{
		switch_state = 0;
		counter = 0;
	}
}

int main(void)
{
	DDRB = 0xFF;
```

```
        PORTD |= (1 << PIND2);
        EIMSK |= (1 << INT0);
        EICRA |= (1 << ISC01);
        sei();
        while(1)
        {
            if(switch_state == 0)
            {
                if(counter == 10)
                {
                    counter = 0;
                }
                PORTB = segments[counter++];
                _delay_ms(500);
            }
        }
    }
```

По такому же принципу я изменил тип переменных, вынес функцию задержки и вывод числа на индикатор за скобки.

Задание 2.

Напишите программу, работающую с кнопкой и двумя 7-сегментными индикаторами, которая будет считать число нажа тий на кнопку до 15 раз.

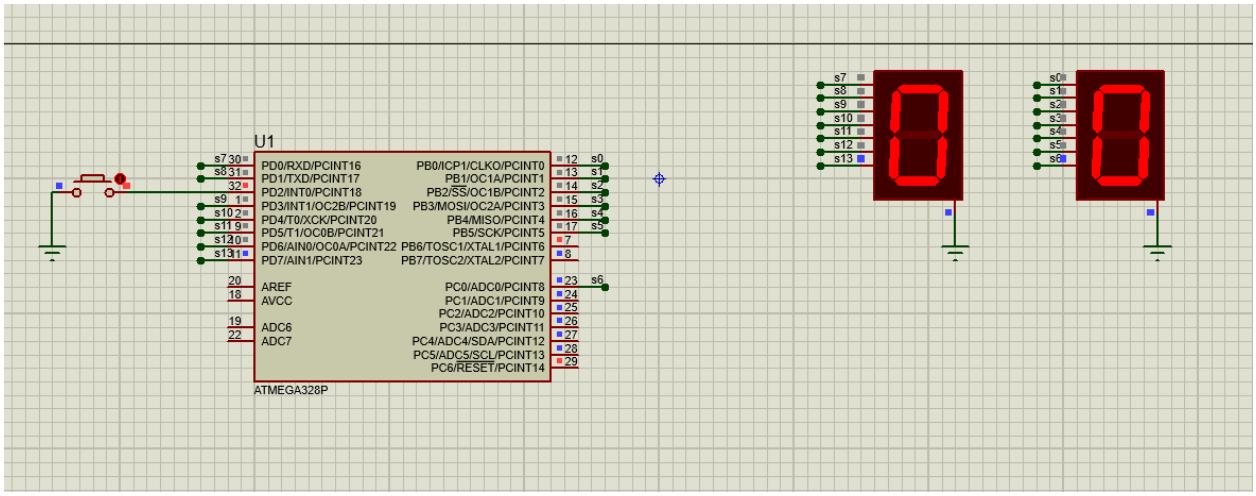
Код программы:

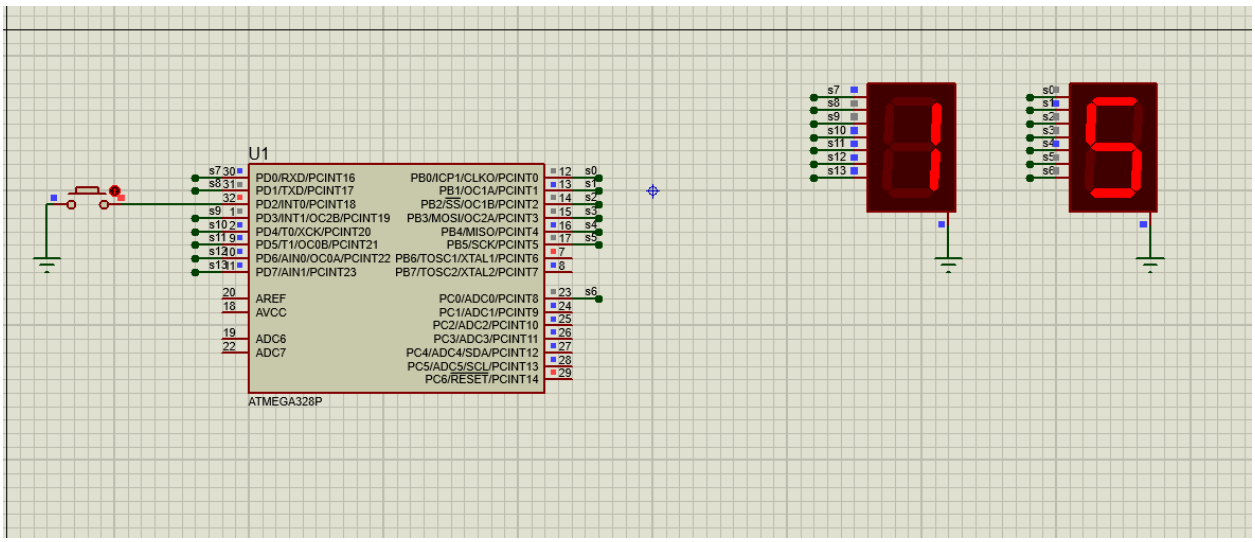
```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

#define MAX_VALUE 16
uint8_t segments[]=
{ //0b01111111
    // ___GFEDCBA
    0b00111111, // 0 - A, B, C, D, E, F
    0b00000110, // 1 - B, C
    0b01011011, // 2 - A, B, D, E, G
    0b01001111, // 3 - A, B, C, D, G
```

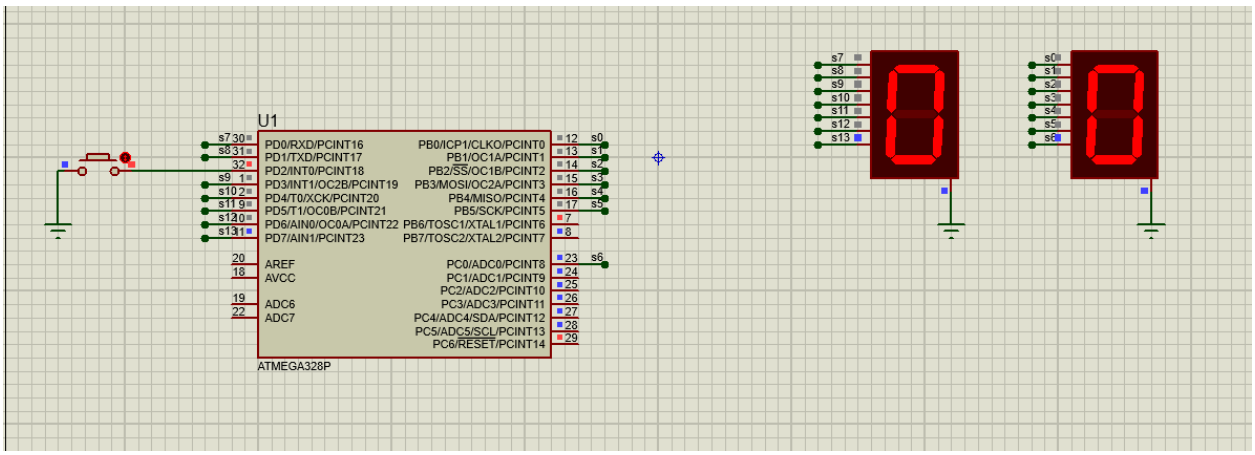
```
        0b01100110, // 4 - B, C, F, G
        0b01101101, // 5 - A, C, D, F, G
        0b01111101, // 6 - A, C, D, E, F, G
        0b00000111, // 7 - A, B, C
        0b01111111, // 8 - A, B, C, D, E, F, G
        0b01101111, // 9 - A, B, C, D, F, G
};
volatile uint8_t switch_state = 0;
volatile uint8_t counter = 0;
volatile uint8_t firstind = 0;
volatile uint8_t secondind = 0;
ISR(INT0_vect)
{
        if(switch_state == 0)
        {
                switch_state = 1;
        }
        else
        {
                switch_state = 0;
        }
        counter++;
}

int main(void)
{

        DDRB = 0xFF;
        DDRC = 0xFF;
        DDRD = 0xFF & ~(1 << PIND2);
        PORTD |= (1 << PIND2);
        EIMSK |= (1 << INT0);
        EICRA |= (1 << ISC01);
        sei();
        while(1)
        {
                if (counter == MAX_VALUE)
                {
                        counter = 0;
                }



                firstind = segments[counter / 10];
                secondind = segments[counter % 10];
```

```
            PORTD = (1 << PIND2);
            PORTB = secondind;
            PORTC = (secondind >> 6);
            PORTD |= firstind & 0b00000011;
            PORTD |= (firstind << 1) & 0b11111000;
            _delay_ms(60);
        }
}
```

Схема в Proteus:







Также, если изменить MAX_VALUE с 16 на 100, то программа также будет работать, и последним числом, выведенным на индикаторы будет 99.

Выводы

Я рассмотрел основные приемы работы с контроллерами AVR. Важными моментами здесь являются работа с портами, сдвиги и битовые операции, прерывания.

Я научился собирать схемы в симуляторе Proteus ISIS.